



Instituto
Telles

Material do Aluno

Técnico em Desenvolvimento *Web* e Cibersegurança

Front-end: princípios

Typescript



Parcerias:

SEDUC
Secretaria de Estado
da Educação

SECTI
Secretaria de
Estado de Ciência,
Tecnologia e Inovação



Sumário

Capítulo 01: O QUE É TYPESCRIPT?	6
1.1 Como surgiu o TypeScript	6
1.2 TypeScript vs JavaScript	7
 Capítulo 02: PRIMEIROS PASSOS NO AMBIENTE TYPESCRIPT	 12
2.1 Ambiente de desenvolvimento	12
2.2 Nosso primeiro código em TypeScript	15
2.3 Configuração do TypeScript	17
 Capítulo 03: CONFIGURAÇÕES E INTEGRAÇÃO COM JAVASCRIPT	 22
3.1 TypeScript Compiler (tsc)	22
3.2 TypeScript em projetos JavaScript	25
 Capítulo 04: COMPILAÇÃO EM TYPESCRIPT	 31
4.1 Mais opções de configuração	31
4.2 Compilação automática e observação	34
 Capítulo 05: TIPOS DE DADOS E VARIÁVEIS	 40
5.1 Mais opções de configuração	40
5.1.1 Dois pilares essenciais: let e const	40
5.2 Dominando os tipos de dados básicos	42
5.2.1 Number x string	42
5.2.2 Valores Lógicos com boolean	44
5.2.3 Booleanos em ação	45
 Capítulo 06: TIPOS DE DADOS, INFERÊNCIA DE DADOS E ARRAYS	 49
6.1 Tipos de dados	49
6.1.1 Desvendando os tipos: anotações e inferência	50
6.1.2 Explorando os tipos em ação	50
6.1.3 Operadores avançados	51
6.1.4 Concatenação de strings	52
6.1.5 Operadores lógicos	53
6.1.6 Arrays em TypeScript: armazenando coleções organizadas	54
6.1.7 Propriedades e métodos úteis	55

Capítulo 07: INTERFACES E TIPOS PERSONALIZADOS EM TYPESCRIPT	58
7.1 Definição de interfaces para descrever a estrutura de objetos	58
7.2 Interfaces personalizadas	60
7.2.1 Aplicação de extends	64
7.2.2 Utilizando tipos	66
7.3 Tipos union	70
7.3.1 Definindo um tipo de união	70
7.4 Tipos intersection	75
7.5 Combinação de tipos com union e intersection	80
 Capítulo 08: DECLARAÇÃO DE CLASSES EM TYPESCRIPT	 87
8.1 Declaração de classes em TypeScript	87
8.2 Propriedades de uma classe em TypeScript	89
8.3 Métodos de uma classes em TypeScript	89
8.4 Modificadores de acesso em TypeScript	93
8.5 Implementação de herança com extends e super	96
8.6 Encapsulamento de dados em TypeScript	99
 Capítulo 09: MÓDULOS E IMPORTAÇÕES EM TYPESCRIPT	 108
9.1 O que são módulos?	108
9.2 Como organizar o código em módulos?	109
9.3 Como importar e exportar um módulo no TypeScript	109
9.4 Resolução de caminhos de módulos com configuração de tsconfig.json	115
9.5 Casos de uso prático	117
 Capítulo 10: GENERICS E EXTENDS NO TYPESCRIPT	 123
10.1 Generics em TypeScript	123
10.2 Aplicações do Generics	125
10.3 Restrições de tipos com extends	130
Referências	139

TYPESCRIPT

INTRODUÇÃO

Bem-vindo à aventura do TypeScript, onde aprenderemos detalhadamente sobre essa ferramenta de desenvolvimento *web*, de uma forma descontraída e prática. Imagine o TypeScript como o super-herói do mundo da programação, oferecendo uma capa extra de proteção ao seu código JavaScript, tornando-o mais seguro e confiável.

Vamos fazer uma analogia simples: você está construindo algo incrível com JavaScript, seja uma casa de LEGO virtual ou uma aplicação *web*. JavaScript é como um parque de diversões cheio de oportunidades emocionantes, mas às vezes você pode tropeçar em alguns erros no meio da diversão. É aí que entra o TypeScript, agindo como um supervisor de construção, garantindo que suas peças se encaixem perfeitamente e que seu código seja mais robusto.

Nesta jornada, vamos explorar como o TypeScript pode ser seu aliado na criação de projetos *web* mais seguros e fáceis de entender. Prepare-se para transformar o complexo em algo compreensível, assim como um amigo de confiança que torna a construção e o desenvolvimento com JavaScript mais emocionante e sem surpresas desagradáveis. Pronto para explorar o universo do TypeScript? Vamos começar!

CAPÍTULO 01

O QUE É TYPESCRIPT?

O que esperar deste capítulo:

- Compreender o que é TypeScript e a sua importância no desenvolvimento web;
- Diferenciar TypeScript e JavaScript.

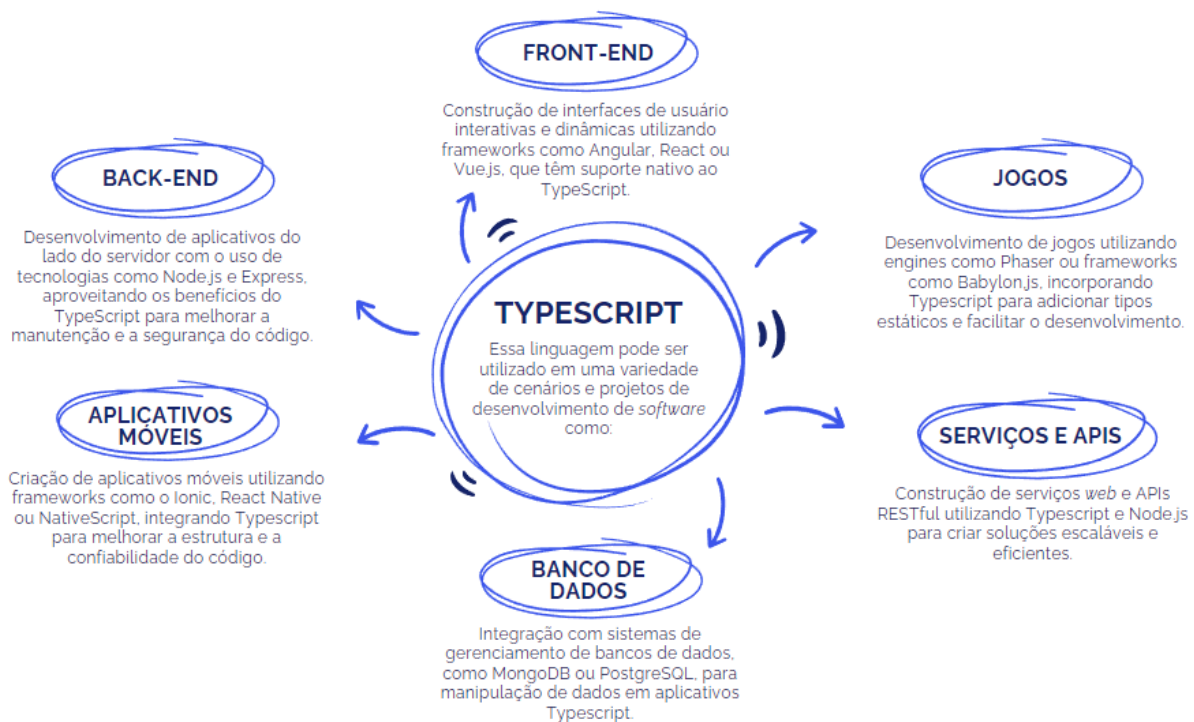
1.1 Como surgiu o TypeScript

No início de 2010, o JavaScript estava se tornando cada vez mais popular e sendo utilizado em diferentes tipos de projetos, desde simples páginas *web* até aplicações mais complexas. No entanto, o JavaScript original não possui algumas características essenciais para o desenvolvimento em grande escala, como um sistema de tipos robusto.

Projetos grandes e complexos escritos somente em JavaScript muitas vezes enfrentavam desafios de manutenção. À medida que a base de código crescia, tornava-se difícil rastrear erros e garantir a consistência do código.

O TypeScript foi desenvolvido pela Microsoft e anunciado publicamente por Anders Hejlsberg, um renomado engenheiro de *software*, durante a conferência Build de 2012. A motivação por trás da criação do TypeScript era melhorar a experiência de desenvolvimento em larga escala no ecossistema JavaScript.

Segundo a pesquisa do StackOverflow de 2022, TypeScript está entre as cinco linguagens mais queridas pelos desenvolvedores e, graças a isso, pode ser encontrada em diferentes tipos de projeto.



1.2 TypeScript vs JavaScript

O TypeScript foi projetado para ser totalmente compatível com o JavaScript existente. Isso significa que o código JavaScript válido também é código TypeScript, permitindo uma transição suave para desenvolvedores que já estavam familiarizados com o ecossistema JavaScript.

A criação do TypeScript proporcionou diversas soluções para o desenvolvimento em JavaScript, como introduzir um sistema de tipos estáticos opcional, permitindo aos desenvolvedores declararem tipos para variáveis, parâmetros de função e valores de retorno de função. Isso ajudou a pegar erros de tipo em tempo de compilação, proporcionando um código mais robusto.

Antes do TypeScript, em JavaScript, poderíamos ter um código assim:

JavaScript

```
function soma(a, b) {  
    return a + b;  
}
```

No entanto, com TypeScript, podemos adicionar tipos para os parâmetros e o retorno:

TypeScript

```
function soma(a: number, b: number): number {  
    return a + b;  
}
```

Essa abordagem permite que os desenvolvedores expressem claramente suas intenções, que, no exemplo do **TypeScript**, **o código só vai aceitar números** enquanto no exemplo de **Javascript**, ele **aceitaria qualquer tipo: texto, número ou booleano**. Isso contribui para que os desenvolvedores detectem mais facilmente os erros antes mesmo de executarem o código.

Escaneie o Qr code a seguir e veja um vídeo explicando a origem dessa linguagem, alguns dos tipos básicos e outras informações, que iremos abordar nos próximos capítulos desta apostila.



Fonte: Vídeo *TypeScript // Dicionário do Programador*, do canal Código Fonte TV. Disponível em: <https://www.youtube.com/watch?v=gmuPEp468IY>. Acesso em: 1 mar. 2024.

A partir do vídeo e dos textos contidos neste capítulo, solucione desafio a seguir.

DESAFIO PRÁTICO

Melhorias no desenvolvimento web com TypeScript.

Descrição

Você está trabalhando na “Guardiões da Web”, uma empresa de desenvolvimento *web* focada na criação de *sites* e sistemas *on-line*. Ela atua há mais de 10 anos no mercado e utiliza JavaScript como linguagem principal para o desenvolvimento de seus sistemas.

Recentemente, a empresa obteve problemas em seus projetos de grande escala e está pensando em adotar uma nova tecnologia para ajudar com as limitações que o JavaScript possui.

Trabalhando em conjunto com seus colegas, expliquem como a adoção de TypeScript pode ajudar a lidar com o problema de escalabilidade, entre outros.

Objetivos

Vantagens de usar TypeScript em comparação com JavaScript puro:

- Explicar as vantagens de usar TypeScript em comparação com JavaScript puro.
- Como o sistema de tipos estáticos do TypeScript pode contribuir para a qualidade e manutenção de um código de *software*?

Tipos de dados:

- Selecionar tipos de dados presentes em TypeScript e exemplificá-los.
-

Orientações

- Apresentar de forma conceitual as diferenças entre TypeScript e JavaScript, destacando os desafios enfrentados pelos desenvolvedores ao escolher entre as duas linguagens;

- Destacar os benefícios específicos do TypeScript, como o sistema de tipos estáticos, e de que forma esses recursos podem abordar desafios comuns encontrados no desenvolvimento de *software* em larga escala;
- Apresentar a jornada de um desenvolvedor ao escolher e implementar o TypeScript, destacando como essa escolha pode influenciar a eficiência e a experiência de desenvolvimento ao longo do tempo.

RESUMO

O TypeScript surgiu da necessidade de aprimorar o desenvolvimento em larga escala no ecossistema JavaScript, introduzindo um sistema de tipos estáticos opcional. Anders Hejlsberg, da Microsoft, liderou o projeto que oferece uma capa extra de proteção ao código, ajudando a evitar erros e melhorar a consistência em projetos complexos. Com exemplos práticos, como a declaração de tipos em funções, o TypeScript proporciona uma codificação mais robusta e facilita a manutenção em longo prazo.

Ao explorar o poder do sistema de tipos estáticos, o TypeScript não apenas aprimora a segurança do código, mas também transforma a experiência de desenvolvimento, promove colaboração eficaz e inspira desenvolvedores a abraçar tecnologias inovadoras para construir um futuro mais promissor no mundo da programação.





ATIVIDADE DE FIXAÇÃO

1. Por que a Microsoft optou por desenvolver o TypeScript?
2. O que é um sistema de tipos estáticos?
3. Explique como o sistema de tipos estáticos do TypeScript pode ser uma vantagem no desenvolvimento de *software*.
4. Quais são as habilidades específicas que os desenvolvedores podem adquirir ao estudar o básico de TypeScript?
5. Como essas habilidades contribuem para um código mais eficiente e seguro?
6. Como o TypeScript facilita a manutenção de projetos de grande escala?
7. Cite três formas de utilizar o TypeScript em projetos de *software*?
8. O que significa o TypeScript ser um superconjunto de JavaScript?
9. Pesquise o porquê de a Microsoft escolher tornar o TypeScript um superconjunto do JavaScript em vez de uma linguagem completamente independente.
10. Como essa decisão impactou a adoção do TypeScript na comunidade de desenvolvimento?

CAPÍTULO 02

PRIMEIROS PASSOS NO AMBIENTE TYPESCRIPT

O que esperar deste capítulo:

- Configurar o ambiente inicial de desenvolvimento;
- Criar e executar um arquivo TypeScript simples;
- Configurar o ambiente TypeScript.

2.1 Ambiente de desenvolvimento

Para rodar TypeScript, você precisará de algumas ferramentas e configurações básicas. A primeira das ferramentas é o Node.js, o qual pode ser instalado através do *link* ou Qr code a seguir.

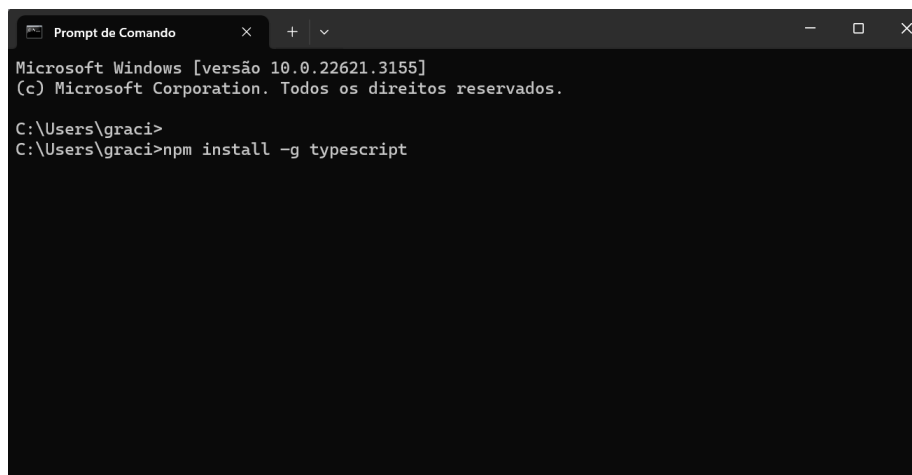


Fonte: <https://nodejs.org/en>

Certifique-se de ter o Node.js instalado, pois ele inclui o npm (*Node Package Manager*), que será usado para instalar o TypeScript e outras dependências. Quando falamos sobre o Node.js no contexto do TypeScript, estamos nos referindo ao uso do Node.js como ambiente de execução para aplicativos TypeScript no lado do servidor.

Pelo fato de o TypeScript ser uma linguagem *superset* (uma linguagem que engloba outra linguagem, contendo todas as suas funcionalidades e adicionando recursos extras) do JavaScript, será necessário instalá-lo para utilizá-lo em seus projetos. Isso pode ser feito executando a seguinte linha de comando em um terminal (por exemplo, Command Prompt para Windows ou Bash para Linux) do seu computador.

```
npm install -g typescript
```



```
Prompt de Comando
Microsoft Windows [versão 10.0.22621.3155]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\graci>
C:\Users\graci>npm install -g typescript
```

NPM é o comando responsável pelo uso do pacote do administrador de pacotes do Node. O comando "install" informa que um novo pacote será instalado, no caso o pacote de TypeScript. E o comando "-g" indica que o TypeScript estará disponível no computador para todos os projetos, isso é, de forma global.

Para as nossas aulas, utilizaremos o TypeScript para criar nossos códigos. Sempre que **iniciarmos um novo projeto**, precisamos **criar a pasta que conterá o nosso projeto** e utilizar o seguinte comando no terminal:

npm init -y

Esse comando irá inicializar o projeto Node.js, autorizar todas as permissões e criar um arquivo "package.json" com as configurações iniciais do projeto.

Após isso, **para instalar o TypeScript no projeto**, precisaremos utilizar o comando:

npm install --save-dev typescript

A opção --save-dev é utilizada para indicar que o pacote TypeScript deve ser adicionado ao campo devDependencies do arquivo package.json.

Os pacotes listados em devDependencies são considerados dependências de desenvolvimento, o que significa que são necessários apenas durante o desenvolvimento e teste do projeto, mas não durante a execução do projeto em produção. Com isso, nossa configuração inicial está concluída e podemos começar a desenvolver códigos em TypeScript.

Lembrando que essa é apenas uma das formas de configurar o ambiente para desenvolvimento em TypeScript. Para aprender outra forma de instalar utilizando o

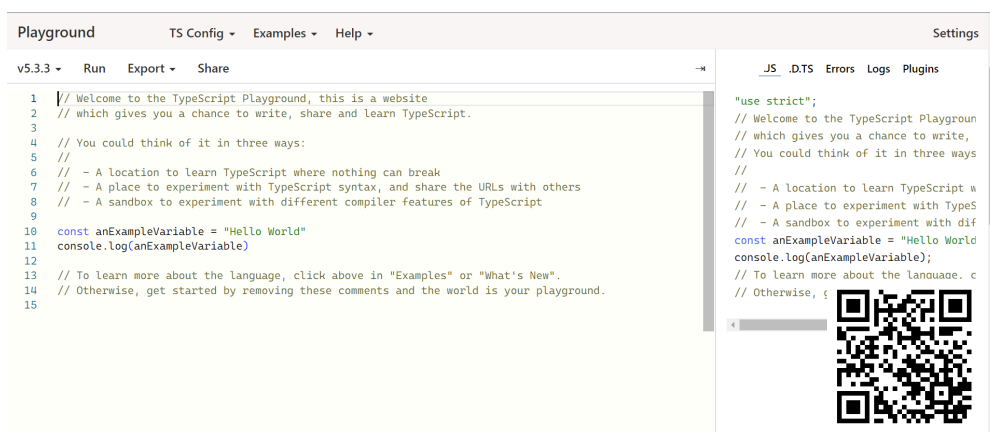
administrador de pacotes Yarn e configurar localmente em seus projetos o TypeScript, veja o vídeo a seguir.



Vídeo *Curso de Typescript - Aula 2: Instalando o Typescript e compilando via TSC*, do canal Daniel Berg. Disponível em: <https://www.youtube.com/watch?v=DSVz-oMpY4k>. Acesso em: 1 mar. 2024.

Vamos explorar uma curiosidade?

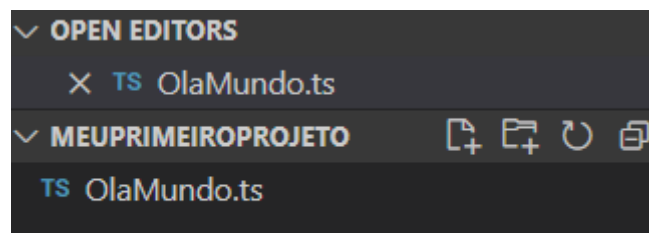
Caso você queira apenas **testar um código TypeScript**, sem se preocupar com as configurações do ambiente em seu computador, você pode usar em qualquer computador, desde que ele possua acesso à internet, o **"Playground"** do próprio **site oficial do TypeScript**. Essa é uma forma bem prática de programar TypeScript em qualquer aparelho que você tiver a disposição.



Fonte: <https://www.typescriptlang.org/pt/play>.

2.2 Nosso primeiro código em TypeScript

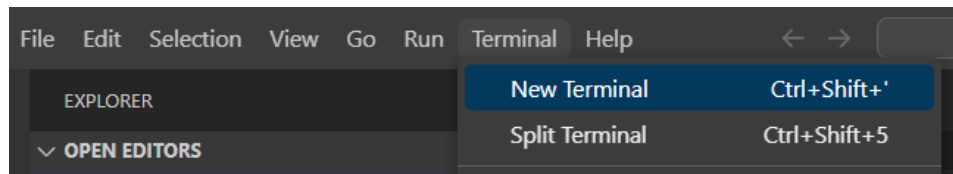
Com nossa configuração inicial terminada, vamos partir para a criação do nosso primeiro Script em TypeScript. Para isso, utilizando o VS Code, **crie uma nova pasta chamada MeuPrimeiroProjeto** e, dentro dela, **um novo arquivo com o nome OlaMundo.ts**. Note que ao acrescentar o "ponto ts" ao final fará com que o arquivo já seja identificado como do tipo TypeScript.



Agora vamos criar nossa primeira linha de código. Como uma boa tradição, os programadores costumam desenvolver uma linha de código que mostre a frase "Olá mundo!" no terminal para celebrar o início da prática de uma nova linguagem, e é dessa forma que vamos começar. Para isso, digite as seguintes linhas de código no seu arquivo e salve ao terminar.

```
TS OlaMundo.ts •
TS OlaMundo.ts > ...
1  let mensagem : string
2
3  mensagem = "Olá mundo!"
4
5  console.log(mensagem)
```

Por fim, para executarmos esse código no nosso projeto, faremos de uma forma simples, utilizando o ambiente que criamos anteriormente com o Node. Basta abrir o terminal presente no VS Code, clicando em **Terminal** e, depois, **New Terminal** (novo terminal).



E, no terminal, executar o comando:

```
npm OlaMundo.ts
```

Ao fazer isso, aparecerá no próprio terminal a mensagem "Olá mundo!". Assim, executamos nosso primeiro programa em TypeScript. Parabéns!

Para praticar a criação de arquivos TypeScript, realize o seguinte desafio prático.

DESAFIO PRÁTICO

Criando códigos em TypeScript.

Descrição

A equipe de gestão da "Guardiões da Web" adorou a explicação sobre o TypeScript que você e sua equipe elaboraram da última vez. Agora, gostariam de ver uma aplicação prática da linguagem. Para isso, pediram que você e a sua equipe criem um código e o executem para que eles vejam a utilização dessa linguagem. Você pode criar um código básico com uma mensagem a ser exibida no terminal.

Objetivos

Compreensão prática do TypeScript:

- Permitir que os estudantes experimentem diretamente a criação e execução de código TypeScript sem a necessidade de compilação explícita.

Familiaridade com ferramentas do ecossistema Node.js:

- Introduzir os estudantes ao ecossistema Node.js e ferramentas relacionadas, como npm e o uso de pacotes do Node.js no desenvolvimento;
- Entender a funcionalidade dos comandos npm e a sua relação com o TypeScript.

Orientações

- Explicar que o objetivo é permitir que eles criem e executem diretamente código TypeScript sem a necessidade de compilação explícita;
- Certificar que os estudantes estejam familiarizados com o ecossistema Node.js;
- Estimular os alunos a explorarem mais a fundo o TypeScript e ferramentas relacionadas, incentivando a curiosidade e a busca contínua por conhecimento.

2.3 Configuração do TypeScript

Imagine que você está planejando uma viagem de férias para um destino incrível. Antes de fazer sua viagem, você precisa fazer alguns preparativos, como comprar passagens, reservar hotéis, preparar as malas, fazer um roteiro da viagem etc. Quando fazemos nossos projetos em TypeScript, também precisamos planejar algumas questões e, para isso, utilizamos o "tsconfig.json", um arquivo fundamental para configurar o TypeScript.

Agora, vamos criar um roteiro nosso! Utilize o **projeto OlaMundo.ts** que criamos anteriormente, **crie um arquivo** com o nome **"tsconfig.json"** na raiz dele e, depois, digite o seguinte código no mesmo arquivo:



```
tsconfig.json X
tsconfig.json > ...
1  {
2      "compilerOptions": {
3          "target": "es5",
4          "module": "commonjs",
5          "strict": true
6      }
7  }
```

- **Target** (alvo): no `tsconfig.json`, o `compilerOptions.target` é como **escolher o destino da sua viagem**. Você decide se está indo para um país que fala inglês, francês ou outro idioma (JavaScript ES5, ES6 etc.). Essa escolha influenciará como você se prepara para a jornada;
- **Module** (módulo): **define como os módulos serão tratados durante a compilação**. O valor `"commonjs"` indica que os módulos serão gerados no formato CommonJS. Imagine que a bagagem que você está levando em sua viagem é como seu código fonte. Dentro da sua bagagem, você pode ter vários compartimentos organizados para diferentes tipos de itens;
- **Strict** (modo estrito): o `compilerOptions.strict` é como **decidir se você quer uma viagem mais controlada e restrita**. Se você ativar o modo estrito, é como decidir ser mais rigoroso consigo mesmo durante a viagem, evitando coisas que poderiam causar problemas.

Para mais informações sobre o `"tsconfig.json"` e as outras opções de configuração, acesse a documentação através do seguinte *link*:



Fonte: <https://www.typescriptlang.org/pt/tsconfig>.

Com a documentação em mãos e o exemplo apresentado anteriormente, realize o seguinte desafio prático para fixação.

DESAFIO PRÁTICO

Configurações do TypeScript.

Descrição

A empresa "Guardiões da Web" começará a utilizar TypeScript para adaptar alguns de seus projetos que eram feitos inteiramente em JavaScript. A equipe de desenvolvimento decidiu adotar o TypeScript para melhorar a qualidade do código e facilitar a manutenção. Para otimizar o ambiente de desenvolvimento, eles estão focando na configuração apropriada do `tsconfig.json`.

Sua missão dessa vez é criar um `tsconfig` para um projeto que foi feito originalmente em JavaScript ES6, utiliza Node.js e precisa passar por verificações. Opções adicionais podem ser utilizadas para integrar o arquivo de configuração.

Objetivos

Melhorias com TypeScript:

- Torna o código mais limpo e organizado. A transição gradual para o modo estrito ajuda a identificar e corrigir potenciais problemas de código.

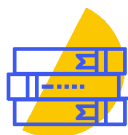
Revisar as configurações:

- Compartilhar aprendizados sobre as vantagens e desafios encontrados durante o processo;
 - Perceber melhorias significativas na qualidade do código e na eficiência do desenvolvimento.
-

Orientações

- Explicar a importância do TypeScript e do `tsconfig.json` no desenvolvimento de projetos web;

- Destacar os desafios enfrentados pela empresa fictícia e como a configuração adequada do `tsconfig.json` pode ajudar a superar esses desafios;
- Iniciar um projeto TypeScript, gerar o `tsconfig.json` padrão e explicar brevemente algumas das opções mais comuns no arquivo.



RESUMO

A configuração inicial do ambiente de desenvolvimento em TypeScript envolve a instalação do TypeScript e a criação de um arquivo `tsconfig.json`. Este arquivo é essencial para definir as configurações do projeto, como o alvo de compilação, diretórios de saída e opções de módulos. A configuração adequada proporciona um ambiente otimizado, permitindo o uso eficaz de tipos e outras funcionalidades do TypeScript durante o desenvolvimento.

Ao criar e executar um arquivo TypeScript simples, os desenvolvedores podem experimentar a transição suave de JavaScript para TypeScript.



ATIVIDADE DE FIXAÇÃO

1. Qual é a importância da configuração inicial do ambiente de desenvolvimento ao trabalhar com TypeScript?
2. Explique os passos necessários para instalar o TypeScript em um ambiente de desenvolvimento.
3. Como você iniciaria a criação de um novo arquivo TypeScript a partir do zero?
4. Quais são os benefícios de se criar um arquivo TypeScript em comparação com um arquivo JavaScript?
5. Qual comando você utilizaria para compilar e executar um arquivo TypeScript no terminal?

6. Como a execução de um arquivo TypeScript difere da execução de um arquivo JavaScript?
7. Cite três configurações que podem ser definidas no arquivo `tsconfig.json`.
8. Explique a função da opção `compilerOptions.target` na configuração do TypeScript.
9. Qual é a diferença entre os valores `"commonjs"` e `"esnext"` para a opção `compilerOptions.module`?
10. Quais são as vantagens de ativar o modo estrito (`strict`) no `tsconfig.json`?

CAPÍTULO 03

CONFIGURAÇÕES E INTEGRAÇÃO COM JAVASCRIPT

O que esperar deste capítulo:

- Configurar o processo de compilação com o TypeScript Compiler (tsc);
- Integrar código TypeScript em projetos JavaScript.

3.1 TypeScript Compiler (tsc)

Configurar o processo de compilação com o TypeScript Compiler (tsc) é uma etapa crucial no desenvolvimento do TypeScript, permitindo traduzir o código TypeScript em código JavaScript para execução.

Imagine que você tem uma coleção de receitas de bolos escritas em um idioma especial. Esse idioma é como o TypeScript, que é uma forma mais detalhada de escrever instruções para fazer bolos. No entanto, os fornos (nossos navegadores ou ambiente de execução) só entendem um idioma mais simples, como o JavaScript.

Vamos refletir?

O TypeScript Compiler (tsc) é como um confeitiro especializado em traduzir suas receitas detalhadas em algo que o forno (navegador) possa entender.

O processo de compilação é como a transformação das receitas escritas, ou seja, a preparação do bolo. Vamos ver isso na prática?



Você pode criar um novo projeto ou utilizar o projeto "OlaMundo.ts" que foi criado no capítulo anterior. Mas, antes de tudo, é necessário instalar o TypeScript globalmente usando o npm (Node Package Manager). Utilize o comando no terminal (Command Prompt para Windows ou Bash para Linux):

```
npm install -g typescript
```

Esse comando instala o TypeScript globalmente, tornando o comando "tsc" disponível no terminal.

O próximo passo é **criar um arquivo** de configuração chamado "**tsconfig.json**" na raiz do projeto. Um arquivo com as opções de configuração padrão pode ser criado automaticamente utilizando o comando:

```
tsc --init
```

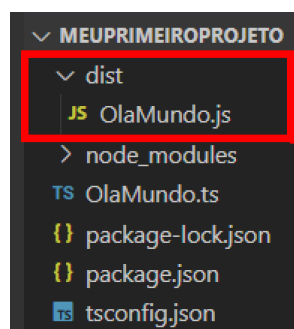
Abra o arquivo gerado e **ajuste as configurações adicionando a seguinte linha:** "**outDir**": "**./dist**". A pasta "dist" é um padrão de boas práticas utilizadas pelos desenvolvedores TypeScript.

A screenshot of a code editor showing the 'tsconfig.json' file. The line '"outDir": "./dist"' is highlighted in green, indicating it has been added or is the focus of the configuration.

O comando "outDir" irá direcionar o arquivo convertido de uma linguagem para outra, ou seja, transpilado em JavaScript para dentro da pasta "dist" que será criada no processo. Para realizar o processo, **abra o terminal e execute o seguinte comando:**

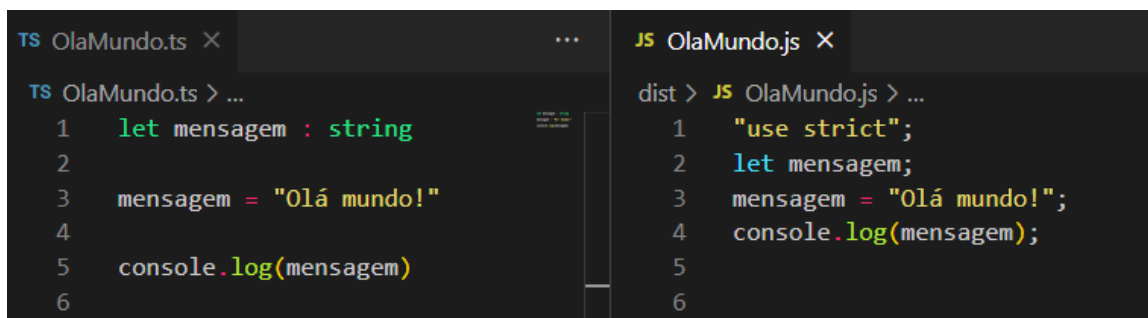
```
tsc OlaMundo.ts
```

Você pode substituir o "OlaMundo.ts" pelo nome do arquivo que estiver usando. Com isso, um novo arquivo surgirá dentro da pasta "dist" com o nome "OlaMundo.js", semelhante a imagem a seguir:



Comparando os dois arquivos, **o código foi transpilado para a versão de JavaScript** descrita nas configurações, alguns outros detalhes podem ser observados como **o tipo das variáveis** e a **linha "use strict"** descrita na configuração do

"tsconfig.json", demonstrado na imagem a seguir:



The image shows a side-by-side comparison of a TypeScript file and its compiled JavaScript output. On the left, the file 'OlaMundo.ts' contains the following code:

```
1 let mensagem : string
2
3 mensagem = "Olá mundo!"
4
5 console.log(mensagem)
6
```

On the right, the file 'OlaMundo.js' in the 'dist' directory shows the compiled JavaScript code:

```
1 "use strict";
2 let mensagem;
3 mensagem = "Olá mundo!";
4 console.log(mensagem);
5
6
```

Essas etapas ilustram como configurar a compilação no ambiente de desenvolvimento com TypeScript. O resultado final será o código JavaScript compilado na pasta "dist", pronto para ser executado em um ambiente Node.js, utilizando o seguinte comando, a partir da raiz do projeto:

```
node ./dist/OlaMundo.js
```

Com isso, será obtida a mensagem "Olá mundo!" no terminal, só que dessa vez, a partir do arquivo em JavaScript. Isso não é incrível?

A seguir veja um vídeo com um exemplo de projeto que utiliza essas mesmas etapas em uma sequência um pouco diferente, realizando primeiro os testes do código e depois o transpilando para JavaScript:



Vídeo *TypeScript - Zero to Hero* | Demo: Primeiro Programa em TypeScript – Hello World #05, do canal Glauca Lemos. Disponível em: <https://www.youtube.com/watch?v=iTCRgdEyaok>. Acesso em: 1 mar. 2024.

A partir do vídeo, realize as seguintes reflexões:

?

Como a configuração do arquivo `tsconfig.json` afeta o comportamento do *TypeScript Compiler* durante o processo de compilação?

?

Qual o propósito da opção `outDir` no arquivo `tsconfig.json` ao configurar o *TypeScript Compiler*?



3.2 TypeScript em projetos JavaScript

Integrar código TypeScript em projetos JavaScript geralmente envolve alguns passos específicos para garantir uma transição suave. O TypeScript é um *superset* do JavaScript, o que significa que o código JavaScript válido também é código TypeScript. No entanto, adicionar TypeScript a um projeto JavaScript permite a introdução de tipos estáticos, interfaces e outros recursos avançados.

Antes de começar o projeto, vamos a um desafio prático para refletir sobre a relação entre TypeScript e JavaScript.



DESAFIO PRÁTICO

Integração de código TypeScript em projetos JavaScript.



Descrição

A equipe de desenvolvimento precisa criar as quatro operações fundamentais de uma calculadora para integrar em seus projetos de cálculo existentes. A você foi dada a tarefa de criar as funções responsáveis por somar, subtrair, dividir e multiplicar. Crie o código em JavaScript para cada uma dessas funcionalidades.

Objetivos

Implementar operações matemáticas básicas:

- Desenvolver funções JavaScript para realizar as operações fundamentais de soma, subtração, multiplicação e divisão;
 - Certificar-se de que as funções manipulem corretamente diferentes tipos de dados numéricos, incluindo inteiros e números de ponto flutuante, mantendo a precisão nos cálculos;
 - Lidar com casos especiais, como divisão por zero, para garantir que as operações sejam seguras e não levem a erros inesperados.
-

Orientações

- Organizar o código de maneira modular, criando uma função separada para cada operação matemática. Isso facilitará a manutenção e a reutilização do código;
 - Adotar boas práticas de codificação, como seguir convenções de nomenclatura, manter o código limpo e legível, e utilizar comentários, quando necessário;
 - Desenvolver o código de forma que seja fácil de entender, modificar e estender. Isso permitirá a integração suave para TypeScript.
-

Agora, podemos criar um projeto básico de calculadora que inicialmente possui apenas código JavaScript ou utilizar o código criado no desafio anterior, e, em seguida, integrar TypeScript. Para começar, vamos criar o arquivo "calculadora.js" na pasta do nosso projeto. Dentro do arquivo JavaScript escreva o seguinte código:

```
JS calculadora.js X
JS calculadora.js > ...
1  function somar(a, b) {
2    |    return a + b;
3  }
4
5  console.log(somar(5, 3));
6
```

Essa é a função base de soma da nossa calculadora, e na linha 5 uma implementação de exemplo da função. Como você pode observar, **não temos a tipagem de nenhum dado**. Para começar a integração, **precisamos instalar o TypeScript no projeto, utilizando os seguintes comandos em um terminal na raiz do projeto**:

```
npm init -y
npm install --save-dev typescript
```

E executar o seguinte comando para gerar um arquivo de configuração TypeScript:

```
npx tsc --init
```

Não esqueça de adicionar a linha a seguir para direcionar os arquivos transpilados:

```
56 // "inlineSourceMap": true,
57 // "outFile": "./",
58 "outDir": "./dist",
59 // "removeComments": true,
```

Para converter o código criado em TypeScript, **renomeie o arquivo "calculadora.js" para "calculadora.ts"**. Os primeiros erros que começaram a ocorrer são relacionados à falta de tipagem e, para resolver isso, atualize o conteúdo com tipagem como no exemplo a seguir:

```
TS calculadora.ts X
TS calculadora.ts > ...
1  function somar(a: number, b: number): number {
2      return a + b;
3  }
4
5  console.log(somar(5, 3));
```

Você tem um projeto JavaScript básico convertido para TypeScript. Este exemplo é bastante simples, mas mostra o processo geral de integração. À medida que seu projeto cresce, você pode explorar mais recursos avançados oferecidos pelo TypeScript.

Agora, vamos realizar um desafio prático utilizando esse mesmo exercício para fixação.

DESAFIO PRÁTICO

Integração de código TypeScript em projetos JavaScript.

Descrição

A equipe de desenvolvimento precisa criar as quatro operações fundamentais de uma calculadora para integrar em seus projetos de cálculo existentes. A você foi dada a tarefa de criar as funções responsáveis por somar, subtrair, dividir e multiplicar. Crie o código TypeScript para cada uma dessas funcionalidades.

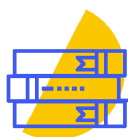
Objetivos

Introduzir TypeScript na aplicação existente:

- Adicionar tipos estáticos para melhorar a detecção de erros durante o desenvolvimento;
 - Melhorar a legibilidade do código e reduzir a complexidade;
 - Capacitar a equipe para continuar o desenvolvimento de novos recursos de maneira mais eficiente.
-

Orientações

- Realizar uma análise estática do código existente para identificar áreas propensas a erros;
 - Começar a introduzir TypeScript em módulos específicos, adicionando tipos gradualmente;
 - Utilizar o TypeScript para modelar os dados e eventos que o componente manipula.
-



RESUMO

O uso eficaz do compilador do TypeScript é essencial para facilitar a integração bem-sucedida do TypeScript em projetos JavaScript, garantindo uma transição suave e aprimorando a qualidade do código ao longo do tempo.

Enquanto isso, integrar código TypeScript em projetos JavaScript envolve uma transição sequencial, começando com a instalação do TypeScript no projeto e a configuração do arquivo `tsconfig.json`. Os arquivos JavaScript podem ser renomeados para `.ts`, permitindo a adição gradual de tipos e a resolução de dependências para bibliotecas externas.



ATIVIDADE DE FIXAÇÃO

1. Como podemos iniciar o processo de integração do TypeScript em um projeto JavaScript?
2. Quais são os passos necessários para configurar o arquivo `tsconfig.json` durante a integração do TypeScript?
3. Explique a importância da resolução de dependências ao adicionar código TypeScript em projetos JavaScript existentes.
4. Qual é o papel do compilador TypeScript (`tsc`) na transição de código JavaScript para TypeScript?
5. Explique como o TypeScript lida com a tipagem estática e como isso beneficia o desenvolvimento em comparação com o JavaScript puro.
6. Quais são os benefícios práticos da integração do TypeScript em projetos JavaScript existentes em termos de manutenção e robustez do código?

7. Quais são os desafios comuns encontrados ao integrar código TypeScript em um projeto JavaScript que apresenta problemas (legado)?
8. Quais são as melhores práticas para garantir uma migração suave de código JavaScript para TypeScript em equipes de desenvolvimento colaborativas?
9. Explique a diferença entre a compilação de arquivos TypeScript individualmente e a compilação de um projeto TypeScript como um todo.
10. Qual é o impacto da integração do TypeScript na performance do aplicativo em comparação com o código JavaScript equivalente?

CAPÍTULO 04

COMPILAÇÃO EM TYPESCRIPT

O que esperar deste capítulo:

- Saber utilizar as opções de configuração do TypeScript para otimizar o desenvolvimento e a integração em projetos JavaScript;
- Compreender a compilação e integração do código TypeScript.

4.1 Mais opções de configuração

Até o momento já vimos algumas configurações básicas do TypeScript para fazer o devido uso desta incrível linguagem de programação através do "**compilerOptions**". Vimos configurações como "**target**", "**module**" e "**strict**", que servem respectivamente para **escolher o destino dos arquivos gerados**, **definir os tipos de módulos gerados** e o **nível de segurança da compilação**.



```
tsconfig.json X
tsconfig.json > ...
1  {
2    "compilerOptions": {
3      "target": "es2020",
4      "module": "commonjs",
5      "strict": true
6    }
7  }
```

Além disso, vimos o "**outDir**", responsável por **determinar a pasta destino dos arquivos TypeScript transpilados**. Mas agora, iremos ver algumas outras configurações que podem ser muito úteis no seu dia a dia de desenvolvimento. Dá uma olhadinha no vídeo a seguir.



Vídeo TypeScript - Zero to Hero | Entendendo o arquivo `tsconfig.json` #04, do canal Gláucia Lemos. Disponível em: <https://l1nq.com/zP8QQ>. Acesso em: 13 mar. 2024.

Agora pense:



Para que serve a configuração `"sourceMap"`?



Quais outras configurações apresentadas no vídeo que você ainda não tinha visto? Para que elas servem?



As **configurações do `tsconfig.json` podem proporcionar flexibilidade** ao adaptar o comportamento do TypeScript às exigências específicas de um projeto, otimizando o desenvolvimento, a manutenção e o desempenho do código.

Vejamos mais algumas delas na prática.

Crie um novo arquivo **`tsconfig.json`** e o **inicialize com as configurações básicas do `"compilerOptions"`**, desta vez, iremos adicionar duas novas configurações **`"outDir"`** e **`"rootDir"`**:

```
tsconfig.json X
tsconfig.json > ...
1  {
2    "compilerOptions": {
3      "target": "es2020",
4      "module": "commonjs",
5      "strict": true
6    },
7    "outDir": "./dist",
8    "rootDir": "./src"
9  }
```


O ***outDir*** define o diretório de saída para os arquivos compilados, enquanto ***rootDir*** indica a raiz do diretório de entrada, ajudando a organizar a estrutura do projeto.

Existem diversas configurações avançadas que podem proporcionar benefícios, incentivar a pesquisa e a leitura da documentação do TypeScript. Faça o seguinte desafio:

DESAFIO PRÁTICO

Configuração avançada.

Descrição

A equipe de desenvolvimento está cada vez mais interessada em conhecer as utilidades do TypeScript e estão decididos a adaptar mais dos seus códigos JavaScript para TypeScript. Para isso, eles precisam entender quais são as configurações necessárias no arquivo *tsconfig.json* para auxiliar nessa transição. Em uma rápida olhada na documentação, descobriram configurações como *esModuleInterop*, *allowJs*, e *checkJs* que podem auxiliar na mudança de linguagem.

Demonstre como essas opções podem ser ajustadas para lidar com código JavaScript existente de forma eficaz e mostre um exemplo prático de utilização.

Objetivos

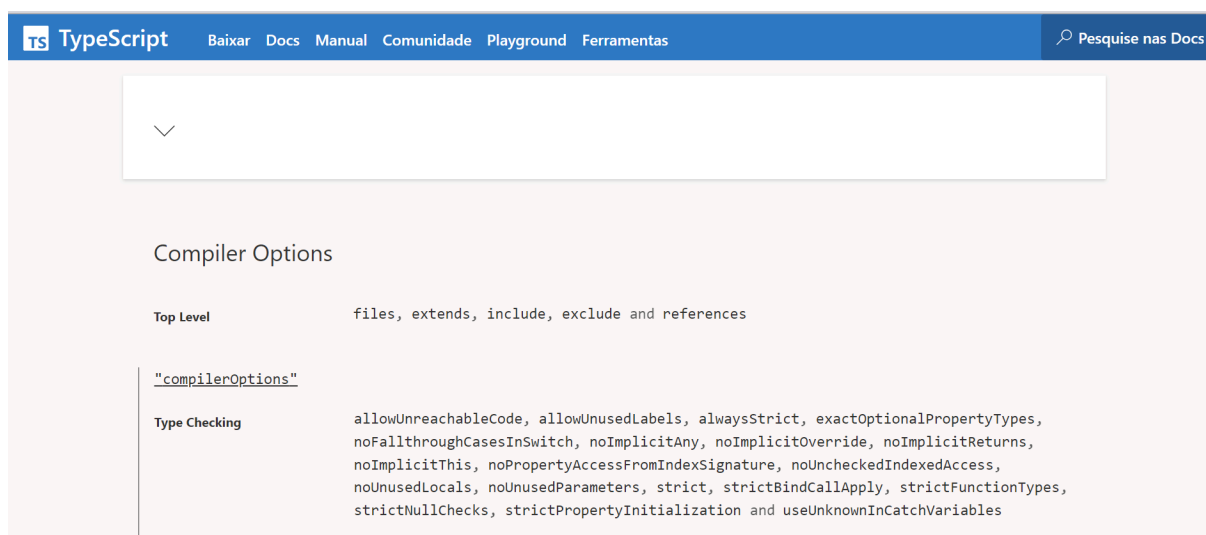
Compreender a importância do *tsconfig* na integração do TypeScript em projetos JavaScript:

- Proporcionar um exercício prático no qual os desenvolvedores possam aplicar os conceitos aprendidos;
- Incentivar a experimentação com diferentes opções de configuração para reforçar o entendimento;
- Adquirir as habilidades necessárias para configurar e otimizar a transição de forma eficiente.

Orientações

- Recapitular os principais pontos da aula, enfatizando a importância da integração gradual do TypeScript;
- Destacar como as opções de configuração do *tsconfig.json* podem ser ajustadas para atender às necessidades específicas do projeto.

Para o auxiliar na pesquisa e criação do arquivo *tsconfig* para esse desafio, consulte a documentação do TypeScript no seguinte link:



Fonte: <https://www.typescriptlang.org/pt/tsconfig>

4.2 Compilação automática e observação

Todas as vezes que um novo arquivo *.ts* é criado, precisamos executar os comandos de transpilação para verificar se todo o processo ocorreu adequadamente. Imagine que, um certo dia, João esqueceu de executar os comandos para transpilar o código, o que acha que isso pode ter causado em seu projeto? Isso gera erros no processo de transpilação.

Pensando neste processo de compilação constante, podemos adicionar um novo comando a nossa lista: ***tsc - watch***. Este comando é **usado para compilar automaticamente o código quando há alterações**. Desta forma, o projeto estará a todo

momento sendo observado pelo TypeScript Compiler. Vamos ver na prática?

No VS Code, **crie uma nova pasta chamada "Soma"** e **execute o seguinte comando no terminal** para instalar o TypeScript:

```
npm install -g typescript
```

Dentro da pasta criada crie um novo arquivo com o nome **app.js** contendo o seguinte código:

```
JS app.js  X
JS app.js > ...
1  function soma(a, b) {
2      return a + b;
3  }
4
5  console.log(soma(5, "10")); // Sem tipos o resultado é imprevisível.
6  // Nesse caso o valor 5 será convertido para texto e os valores serão concatenados,
7  // resultando no texto "510".
```

Vamos **renomear esse arquivo para app.ts** e **adicionar tipos** para ver a diferença:

```
TS app.ts  X
TS app.ts > ...
1  function soma(a: number, b: number): number {
2      return a + b;
3  }
4
5  console.log(soma(5, 10)); // Agora, com os tipos definidos corretamente
6  // o resultado se torna mais previsível, sendo a soma de dois números.
```

O uso de tipos torna o processo mais seguro e mais fácil detectar erros durante o desenvolvimento. Crie um novo arquivo **tsconfig.ts** usando o seguinte comando no terminal:

```
tsc --init
```

E para iniciar o processo de observação e compilação automática, **utilize o comando watch** no terminal:

```
tsc --watch
```

A seguinte tela aparecerá no seu terminal indicando que o código está sendo compilado em tempo real automaticamente:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[14:47:12] File change detected. Starting incremental compilation...

[14:47:12] Found 0 errors. Watching for file changes.
```

A fim de testar a funcionalidade de observação vamos fazer uma pequena alteração no código e salvá-la. Ao fazer isso, o compilador irá mostrar todos os erros resultados dessa modificação:

```
TS app.ts 1 X
TS app.ts > soma
1 function soma(a, b: number): number { //Retirando o tipo de um dos argumentos, um erro é apontado no terminal.
2   return a + b;
3 }
4
5 console.log(soma(5, 10)); // Agora, com os tipos definidos corretamente
6 // o resultado se torna mais previsível, sendo a soma de dois números.
7

PROBLEMS 1 OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[14:51:57] File change detected. Starting incremental compilation...

app.ts:1:15 - error TS7006: Parameter 'a' implicitly has an 'any' type.

1 function soma(a, b: number): number { //Retirando o tipo de um dos argumentos, um erro é apontado no terminal.
    ~~~~~

[14:51:57] Found 1 error. Watching for file changes.
```

O erro indica que existe um tipo implícito para a variável "a", que nesse caso é o tipo *any*. Esse tipo não seria seguro e adequado para esse caso específico, por isso é apontado o erro e o código não é transpilado. **Devolva o tipo *number* para a variável "a" para consertar o código.**

Agora, para praticar esse conceito de compilação, resolva o seguinte desafio para fixação:

DESAFIO PRÁTICO

Compilando automaticamente.

Descrição

As preparações estão quase prontas e sua equipe de desenvolvimento está cada dia mais engajada em utilizar TypeScript no desenvolvimento de seus programas, principalmente ao verem tantos benefícios em sua utilização. Crie, em TypeScript, um programa que recebe um número e dobra o seu valor, mostrando o valor no final.

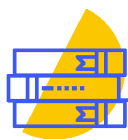
Objetivos

Compreender a utilização da compilação automática para facilitar o desenvolvimento com TypeScript:

- Integrar gradualmente o TypeScript em projetos JavaScript;
 - Adicionar tipos de forma progressiva;
 - Configurar opções avançadas para otimizar o desenvolvimento.
-

Orientações

- Utilizar os comandos `watch` para a compilação automática e `init` para a criação automática do arquivo `tsconfig.json`.
 - Destacar como as opções de configuração do `tsconfig.json` podem ser ajustadas para atender às necessidades específicas do projeto.
-



RESUMO

As configurações avançadas do TypeScript no arquivo *tsconfig.json* permitem ajustar o comportamento do compilador de acordo com as necessidades específicas do projeto. Algumas opções importantes incluem os itens a seguir.

- ***esModuleInterop***: facilita a interoperabilidade entre módulos *CommonJS* e ES6, resolvendo problemas com a importação/exportação de módulos;
- ***allowJs*** e ***checkJs***: ***allowJs*** permite a inclusão de arquivos JavaScript no projeto enquanto ***checkJs*** habilita a verificação de tipos nesses arquivos, permitindo uma transição gradual de JavaScript para TypeScript;
- ***sourceMap***: gera arquivos de mapa de origem (.map), simplificando a depuração ao mapear o código TypeScript para o código JavaScript transpilado.

Enquanto a compilação automática reduz o tempo entre a edição do código e a visualização dos resultados, aumenta a produtividade e contribui para a detecção precoce de erros.



ATIVIDADE DE FIXAÇÃO

1. Qual é o propósito da opção *esModuleInterop* no arquivo *tsconfig.json* e como ela facilita a interoperabilidade (capacidade de um sistema de se comunicar com outro) entre diferentes sistemas de módulos?
2. Explique como as opções *allowJs* e *checkJs* podem ser utilizadas para incorporar gradualmente arquivos JavaScript em um projeto TypeScript, destacando seus efeitos.
3. Por que a geração de *source maps* (opção *sourceMap*) é benéfica durante o desenvolvimento? Como ela facilita a depuração em projetos TypeScript?
4. Como a configuração *strict* em *compilerOptions* pode impactar a rigidez na verificação de tipos e na escrita de código seguro em TypeScript?

5. Em que cenários práticos a opção *noEmitOnError* pode ser útil e como ela afeta o processo de compilação?
6. O que significa usar o comando *tsc --watch* e como ele facilita o desenvolvimento contínuo em projetos TypeScript?
7. Quais são os benefícios de integrar o *--watch* em *scripts* de construção (por exemplo, no arquivo *package.json*) em termos de automação durante o desenvolvimento?
8. Como a compilação automática contribui para a identificação precoce de erros de código e por que isso é importante no ciclo de desenvolvimento?
9. Em que medida a compilação automática pode afetar a eficiência do desenvolvimento, reduzindo o tempo entre a edição do código e a visualização dos resultados?
10. Explique como a compilação automática pode ser ajustada ou configurada de acordo com as necessidades específicas de um projeto TypeScript, considerando opções avançadas.

CAPÍTULO 05

TIPOS DE DADOS E VARIÁVEIS

O que esperar deste capítulo:

- Compreender a declaração de variáveis com *let* e *const*;
- Diferenciar os tipos de dados básicos, como *number*, *string* e *boolean*.

5.1 Mais opções de configuração

Em sua essência, uma **variável** é um **compartimento virtual na memória do computador**, utilizado para armazenar dados durante a execução do programa. Imagine-a como uma **caixa mágica** que **guarda informações de diferentes tipos**, como **números**, **textos** ou **valores lógicos**. Ela recebe um nome único, que serve como referência para acessá-la e manipular seus dados.

5.1.1 Dois pilares essenciais: *let* e *const*

Ao declarar variáveis em TypeScript, você tem a seu dispor duas ferramentas poderosas: *let* e *const*. Cada uma delas possui características únicas que moldam o comportamento e a utilidade da variável.

A palavra-chave ***let*** permite declarar variáveis com escopo de bloco, ou seja, elas só são acessíveis dentro do bloco de código onde foram criadas. Isso garante organização e previsibilidade no código, evitando conflitos de nomes e variáveis "fantasmas".

Variáveis declaradas com ***let*** podem ser alteradas livremente ao longo do programa, oferecendo flexibilidade para manipular seus valores de acordo com a lógica do seu código.

Exemplos:

```
let nome: string = "João"; // Declaração e inicialização com valor "João"  
console.log(nome); // "João"  
// Modificando o valor da variável  
nome = "Maria"; // Reatribuição
```



```
console.log(nome); // "Maria"
```

```
if (true) {
```

```
  let idade: number = 18;
```

```
  console.log(idade); // 18
```

```
}
```

```
// 'idade' não está acessível aqui, pois está dentro do bloco 'if'
```

A palavra-chave **const** introduz variáveis constantes, que armazenam valores imutáveis após a inicialização. Isso garante segurança e confiabilidade no código, evitando alterações acidentais em dados importantes. Uma vez definida, uma variável *const* não pode ser modificada, protegendo a integridade do seu código e prevenindo erros indesejados.

Exemplos:

```
const PI: number = 3.14159; // Declaração e inicialização com valor constante
```

```
console.log(PI); // 3.14159
```

```
// Erro! Tentativa de reatribuição
```

```
// PI = 3.14; // Gera um erro de compilação
```

```
const usuario = {
```

```
  nome: "Ana",
```

```
  idade: 20,
```

```
};
```

```
// Modificar propriedades do objeto é permitido, mas a referência 'usuario' permanece constante
```

```
usuario.idade = 21;
```

```
console.log(usuario.idade); // 21
```

```
// Erro! Tentativa de reatribuir a referência 'usuario'
```

```
// usuario = { nome: "João", idade: 30 }; // Gera um erro de compilação
```

Para saber mais:



The screenshot shows the TypeScript Handbook website. The left sidebar has a menu with items like 'Get Started', 'Handbook', 'Reference', 'Tipos Utilitários', 'Cheat Sheets', 'Decoradores', 'Fusão de Declarações', 'Enums', 'Iteradores e Geradores', 'JSX', and 'Mixins'. The 'Reference' item is selected. The main content area is titled 'Declarações de variáveis'. The text explains that `let` and `const` are relatively new concepts for variable declarations in JavaScript. It states that `let` is similar to `var` in some aspects but avoids some of its pitfalls. It also mentions that `const` is an extension of `let` that prevents reassignment. The text concludes by noting that TypeScript, as an extension of JavaScript, naturally supports `let` and `const`, and that they have better preference than `var`.

Fonte: <https://www.typescriptlang.org/pt/docs/handbook/variable-declarations.html>

5.2 Dominando os tipos de dados básicos

Em *TypeScript*, cada variável possui um tipo de dado que define o tipo de informação que ela pode armazenar. O uso correto dos tipos de dados garante a confiabilidade do seu código, prevenindo erros de conversão e inconsistências. Exploreemos os tipos básicos mais utilizados:

5.2.1 Number x string



Exemplo de *number*:

```
let numero: number = 10; // Declaração de uma variável 'numero' do tipo 'number'
let outroNumero: number = 5.25;
let soma = numero + outroNumero; // 15.25
let diferenca = numero - outroNumero; // 4.75
let multiplicacao = numero * outroNumero;
```

Exemplos de *string*:

```
let nome: string = "Alice"; // Declaração de uma variável 'nome' do tipo 'string'
let mensagem: string = "Olá, mundo!";
let saudacao = "Bem-vindo, " + nome; // Concatenação de strings
console.log(saudacao); // "Bem-vindo, Alice"
let primeiroCaractere = mensagem[0]; // Extraíndo o primeiro caractere ('O')
console.log(primeiroCaractere);
```

```
// Comparação de strings (sensível a maiúsculas e minúsculas)
let resultado = nome === "alice"; // false (maiúsculas diferentes)
console.log(resultado);
```

```
// Conversão de string para number (se possível)
let numeroString = "123";
let numeroConvertido = Number(numeroString); // 123 (convertido com sucesso)
console.log(numeroConvertido);
```

Para saber mais:



Vídeo TypeScript - Zero to Hero | Demo - Tipo String #14, do canal Gláucia Lemos. Disponível em: <https://encr.pw/lcbLJ>. Acesso em: 8 mar. 2024.

5.2.2 Valores Lógicos com boolean

O **boolean** representa valores lógicos que podem ser apenas **true** (verdadeiro) ou **false** (falso). Ele permite realizar operações lógicas como **&&** (AND), **||** (OR), **!** (NOT) para combinar e negar valores booleanos, além de serem **utilizadas em condicionais e loops**.

Imagine um interruptor de luz. Quando ele está ligado, a luz acende, representando o valor "**verdadeiro**" (*true*). Quando está desligado, a luz apaga, representando o valor "**falso**" (*false*). Na programação, os booleanos funcionam da mesma maneira.

Exemplos:

```
let isMaiorDeldade: boolean = false; // Declaração de uma variável 'isMaiorDeldade' do tipo 'boolean'
```

```
let estaChovendo: boolean = true;
```

```
let temPermissao = isMaiorDeldade && estaChovendo; // Operador AND (ambas as condições devem ser verdadeiras)
```

```
console.log(temPermissao); // false
```

```
let podeSair = !estaChovendo || isMaiorDeldade; // Operador OR (pelo menos uma condição deve ser verdadeira)
```

```
console.log(podeSair); // true
```

```
if (isMaiorDeldade) {
```

```
    console.log("Você é maior de idade.");
```

```
} else {
```

```
    console.log("Você não é maior de idade.");
```

```
}
```

Operações booleanas

E (&&)	Ou ()	Não (!)
Imagine dois interruptores, para a luz acender, ambos os interruptores precisam estar ligados. Em termos booleanos, isso significa que ambas as condições precisam ser verdadeiras para que o resultado seja verdadeiro. Exemplo: se o usuário digitou a senha correta (verdadeiro) "E" se o usuário é maior de idade (verdadeiro), então permitir o acesso ao sistema (verdadeiro).	Imagine um dos dois interruptores. Se qualquer um dos interruptores estiver ligado, a luz acende. Em termos booleanos, isso significa que pelo menos uma das condições precisa ser verdadeira para que o resultado seja verdadeiro. Exemplo: se o usuário digitou a senha correta (verdadeiro) "OU" se o usuário é administrador do sistema (verdadeiro), então permitir o acesso ao sistema (verdadeiro).	Imagine um interruptor que inverte o estado da luz. Se a luz estiver acesa, ele a apaga. Se a luz estiver apagada, ele a acende. Em termos booleanos, o operador "não" inverte o valor de uma condição. Exemplo: se o usuário não digitou a senha correta (verdadeiro), então negar o acesso ao sistema (verdadeiro).

5.2.3 Booleanos em ação

Condicionais: imagine um guarda em um castelo. Ele só permite a entrada de pessoas que são "cavaleiros" OU "princesas".

```
if (usuario é cavaleiro || usuario é princesa) {  
    permitir entrada no castelo;  
} else {  
    negar entrada no castelo;  
}
```

Loops: imagine um robô que precisa coletar 10 moedas para completar uma missão.

```
while (numeroDeMoedas < 10) {  
    procurar e coletar moedas;  
    aumentar numeroDeMoedas;  
}
```

Booleanos são ferramentas essenciais para controlar o fluxo de seus programas e tomar decisões importantes. Dominar os operadores lógicos e utilizá-los em condicionais e loops permite que você crie programas mais complexos e interativos.

Para saber mais:



Video *TypeScript - Zero to Hero* | *Demo - Boolean #10*, do canal *Glaucia Lemos*. Disponível em: <https://lnk.dev/BqSie>.

Acesso em: 8 mar. 2024.



Vimos que em *TypeScript* as variáveis são a base para armazenar dados durante a execução do programa. Ao declarar uma variável, você define o seu nome, tipo de dado e valor inicial.

As variáveis podem ser declaradas com ***let*** ou ***const***. *Let* permite criar variáveis com escopo de bloco, acessíveis apenas dentro do bloco onde foram declaradas, sendo possível modificar o valor a variável ao longo do programa. Já *const* cria variáveis constantes, com valor imutável após a inicialização, sendo ela ideal para valores que não devem ser alterados, como constantes matemáticas ou URLs.

Os tipos de dados básicos em *TypeScript* são ***number***, ***string*** e ***boolean***. O *number* representa números inteiros (ex: 10) ou fracionários (ex: 3.14) e nos permite realizar cálculos matemáticos e operações complexas. Já o *string* representa sequências de caracteres, como palavras, frases ou textos completos, com esse tipo, podemos concatenar strings, extrair caracteres e realizar comparações. E por fim, o *boolean* representa valores lógicos, apenas *true* (verdadeiro) ou *false* (falso), nos permitindo realizar operações lógicas e é muito utilizada em dados condicionais e *loops*.



ATIVIDADE DE FIXAÇÃO

1. Como se declara uma variável em *TypeScript*?
 - a. com a palavra-chave *"variable"*.
 - b. com a palavra-chave *"var"*.
 - c. com a palavra-chave *"let"*.
 - d. com a palavra-chave *"const"*.

2. Qual palavra-chave é utilizada para criar variáveis com escopo de bloco em *TypeScript*?
 - a. *var*.
 - b. *let*.
 - c. *const*.
 - d. *block*.

3. Qual é a principal diferença entre as variáveis criadas com *let* e *const* em *TypeScript*?
 - a. Variáveis criadas com *let* não podem ser modificadas enquanto as criadas com *const* podem.
 - b. Variáveis criadas com *const* não podem ser modificadas enquanto as criadas com *let* podem.
 - c. Não há diferença entre elas.
 - d. Variáveis criadas com *let* têm escopo global enquanto as criadas com *const* têm escopo de bloco.

4. Qual é o tipo de dado utilizado para representar números inteiros e fracionários em *TypeScript*?
 - a. *number*.
 - b. *integer*.
 - c. *float*.
 - d. *numeric*.

5. Como se declara uma variável constante em *TypeScript*?
 - a. utilizando a palavra-chave *"const"*.
 - b. utilizando a palavra-chave *"let"*.
 - c. utilizando a palavra-chave *"constant"*.
 - d. utilizando a palavra-chave *"immutable"*.
6. Qual é o tipo de dado utilizado para representar sequências de caracteres em *TypeScript*?
 - a. *string*.
 - b. *text*.
 - c. *character*.
 - d. *sequence*.
7. Escreva um exemplo de código em *TypeScript* que declare uma variável do tipo *number* e atribua o valor 10 a ela.
8. Escreva um exemplo de código em *TypeScript* que declare uma variável constante chamada "PI" e atribua o valor de 3.14 a ela.
9. Escreva um exemplo de código em *TypeScript* que declare uma variável do tipo *string* e atribua o valor "Olá, mundo!" a ela.
10. Escreva um exemplo de código em *TypeScript* que declare uma variável do tipo *boolean* e atribua o valor *true* a ela.

CAPÍTULO 06

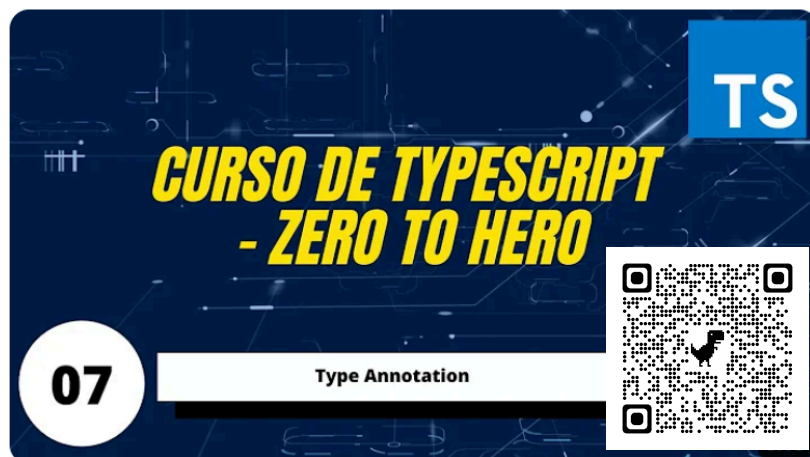
TIPOS DE DADOS, INFERÊNCIA DE DADOS E ARRAYS

O que esperar deste capítulo:

- Compreender o uso de anotações de tipo para especificar tipos de variáveis;
- Saber realizar a inferência de tipos em TypeScript.
- Conhecer a manipulação e utilização de *arrays*.

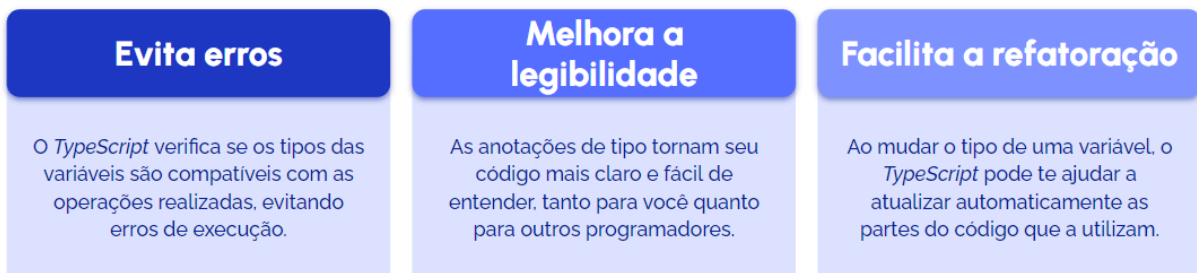
6.1 Tipos de dados

Imagine que um cozinheiro experiente está preparando um delicioso banquete. O que ele precisa fazer? Escolher os ingredientes para cada receita é um passo inicial, não acha? Assim como ele precisa dos ingredientes certos para cada receita, em *TypeScript* você precisa definir os tipos de suas variáveis para garantir que seu código seja preciso e confiável. Os tipos em *TypeScript* são como etiquetas que você coloca nas suas variáveis, informando qual tipo de dado elas armazenam. Isso ajuda a evitar erros e torna seu código mais fácil de entender.



Video *TypeScript - Zero to Hero* | *Type Annotation* #07, do canal Glauca Lemos. Disponível em: <https://acesse.dev/8GSFY>. Acesso em: 8 mar. 2024.

Benefícios de usar tipos



6.1.1 Desvendando os tipos: anotações e inferência

Anotações de tipo

As anotações de tipo ajudam a especificar as suas variáveis, informando ao *TypeScript* qual tipo de dado elas armazenam. Isso ajuda a evitar erros e torna seu código mais fácil de entender.

Exemplo:

```
let nome: string = "João"; // Anotação de tipo "string" para a variável "nome"
```

```
let idade: number = 17; // Anotação de tipo "number" para a variável "idade"
```

Inferência de tipos

Em muitos casos, o *TypeScript* pode inferir o tipo de uma variável automaticamente, observando o valor que você atribui a ela. Isso torna seu código mais conciso e facilita a escrita.

Exemplo:

```
let nome = "Maria"; // Inferência de tipo "string" para a variável "nome"
```

```
let idade = 18; // Inferência de tipo "number" para a variável "idade"
```

6.1.2 Explorando os tipos em ação

Operações matemáticas

Em *TypeScript*, você pode realizar diversas operações matemáticas com os tipos de dados numéricos, como `number` e `bigint`. As operações básicas incluem:



Exemplo:

```
let num1: number = 10;
```

```
let num2: number = 5;
```

```
let soma = num1 + num2; // 15
```

```
let diferenca = num1 - num2; // 5
```

```
let multiplicacao = num1 * num2; // 50
```

```
let divisao = num1 / num2; // 2
```

```
let modulo = num1 % num2; // 0
```

```
console.log(soma, diferenca, multiplicacao, divisao, modulo);
```

6.1.3 Operadores avançados

O TypeScript também oferece operadores matemáticos mais avançados, como:



Exemplo:

```
let num1: number = 10;
```

```
let num2: number = 2;
```

```
let exponenciacao = num1 ** num2; // 100
```

```
let raizQuadrada = Math.sqrt(num1); // 3.162277660168379
```

```
let raizCubica = Math.cbrt(num1); // 2.1544346900318837
```

```
let log = Math.log(num1); // 2.302585092994046
```

```
let arredondamento = Math.round(num1); // 10
```

```
console.log(exponenciacao, raizQuadrada, raizCubica, log, arredondamento);
```

6.1.4 Concatenação de *strings*

A concatenação acontece quando usamos o símbolo + entre **duas ou mais variáveis** no código, **unindo os valores dentro de uma terceira variável**.

Exemplo:

```
let mensagem: string = "Olá, "; // Anotação de tipo "string"
```

```
let nome: string = "Ana"; // Anotação de tipo "string"
```

```
let saudacao = mensagem + nome; // A concatenação de duas strings resulta em uma
"string"
console.log(saudacao); // Olá, Ana
```

6.1.5 Operadores lógicos

Em TypeScript, os operadores lógicos são ferramentas usadas para combinar expressões booleanas (*true* ou *false*) e controlar o fluxo do seu código. Eles permitem construir condições complexas e tomar decisões baseadas em múltiplos fatores.

Tipos de operadores lógicos:

Boolean	E comercial (&&)
Podem ter <i>true</i> ou <i>false</i> .	Verifica se ambas as expressões são verdadeiras.
Ou comercial ()	Negação (!)
Verifica se pelo menos uma das expressões é verdadeira.	Inverte o valor booleano de uma expressão (<i>true</i> vira <i>false</i> e vice-versa).

Exemplos:

```
let isAluno: boolean = true; // Anotação de tipo "boolean"
```

```
let estaPresente: boolean = false; // Anotação de tipo "boolean"
```

```
let podeEntrar = isAluno && estaPresente; // O operador "&&" combina duas condições
"boolean"
```

```
console.log(podeEntrar); // false
```

```
let podeSair = !estaPresente || isAluno; // O operador "||" combina duas condições
"boolean"
```

```
console.log(podeSair); // true
```

Vamos ver um exemplo de uso desses operadores lógicos? O código abaixo verifica se, de acordo com a idade, a pessoa é maior ou menor de 18 anos.

Verificando Maioridade:

```
let idade: number = 18;
```

```
let isMaiorDeldade: boolean = idade >= 18; // >= compara se é maior ou igual dentro de uma variável boolean.
```

```
if (isMaiorDeldade) {  
    console.log("Você é maior de idade!");  
} else {  
    console.log("Você ainda não é maior de idade.");  
}
```

6.1.6 Arrays em TypeScript: armazenando coleções organizadas

Imagine uma caixa de ferramentas com compartimentos separados para cada ferramenta. Cada compartimento guarda uma coisa, como prego, parafuso e martelo. Em *TypeScript*, os *arrays* funcionam de forma similar, permitindo armazenar uma coleção ordenada de elementos do mesmo tipo que podem ser acessados durante o código.

Um *array* é uma estrutura de dados que agrupa elementos sob um único nome. Cada elemento dentro do *array* possui uma posição específica, identificada por um índice que começa em zero.

Para declarar um *array*, você precisa especificar o tipo de dado que ele irá armazenar e o número de elementos (opcional).

Existem duas formas comuns de declarar *arrays*:

Forma 1: Usando o tipo genérico `Array<tipo>`:

```
let frutas: Array<string> = ["maçã", "banana", "laranja"];
```

Forma 2: Usando colchetes `[]` e o tipo dos elementos:

```
let numeros: number[] = [1, 2, 3, 4, 5];
```

Acessando Elementos:

Você pode acessar elementos individuais de um *array* usando o índice entre colchetes `[]`.

let primeiraFruta = frutas[0]; // Acessa o primeiro elemento ("maçã")

let ultimoNumero = numeros[numeros.length - 1]; // Acessa o último elemento (5)

Ao acessar um *array*, a contagem dos elementos começa em 0 (chamamos essa posição de índice). Sendo assim, no exemplo de frutas, maçã está no índice 0, banana está no 1 e laranja no 2.

6.1.7 Propriedades e métodos úteis



Quer saber um pouco mais? Confere esse vídeo.



Vídeo #02 - Typescript. Tipagens de variáveis, parâmetros e retornos de métodos ou funções

Fonte:

<https://www.youtube.com/watch?v=-M6PZl1aXFo>



RESUMO

O TypeScript é uma linguagem de programação que **adiciona tipagem estática ao JavaScript**, tornando o código mais preciso e confiável. As variáveis em TypeScript podem armazenar diferentes tipos de dados, como **números (*number*)**, **textos (*string*)**, **valores booleanos (*boolean*)**, **objetos (*object*)** e o **tipo genérico *any***.

É possível **definir explicitamente o tipo de uma variável** em TypeScript, como em **"let nome: string"**. Isso **aumenta a segurança e legibilidade do código**, pois o TypeScript verifica se os tipos das variáveis são compatíveis com as operações realizadas, evitando erros e facilitando a refatoração.

O TypeScript também pode inferir o tipo de uma variável automaticamente, observando o valor que você atribui a ela. Isso torna o código mais conciso, como em **"let idade = 18"**.



ATIVIDADE DE FIXAÇÃO

1. As variáveis armazenam dados específicos de acordo com seu tipo. Se for preciso armazenar nomes de pessoas, qual tipo de variável é mais adequado?
 - a. ``number``.
 - b. ``string``.
 - c. ``boolean``.
 - d. ``object``.
2. O código abaixo determina de acordo com a variável `idade` se uma pessoa é ou não maior de idade. Complemente o código a partir das duas barras `///
//` para que seja exibido o resultado no final.

```
```typescript
```

```
let idade: number = 20;
```

```
if (idade > 18) {
```

```
 console.log("Você é maior de idade.");
```



```

} else {
 console.log("Você é menor de idade.");
}

// ...

```

3. Qual é a diferença entre anotações de tipo e inferência de tipos em TypeScript?
4. Escreva um programa que calcula a área de um retângulo.
5. Crie um *array* que armazena os nomes de três frutas diferentes.
6. Qual operador é usado para verificar se um valor é igual a outro em TypeScript?
  - a. ``+``
  - b. ``-``
  - c. ``*``
  - d. ``==``
7. O código abaixo faz a somatória das variáveis `numero1` e `numero2`. Escreva o que deveria ter em `//` para que o programa funcione:

```

``typescript
let numero1: number = 10;
let numero2: number = 5;

let resultado: number = //;

console.log("A soma de " + numero1 + " e " + numero2 + " é igual a " + resultado);

```

8. Explique como usar o tipo ``any`` em TypeScript.
9. Escreva um programa que verifica se um número é par ou ímpar.
10. Crie um *array* que armazena as notas de um aluno em três provas.

# CAPÍTULO 07

## INTERFACES E TIPOS PERSONALIZADOS EM TYPESCRIPT

### O que esperar deste capítulo:

- Verificar a funcionalidade do código, executando testes e sua depuração;
- Compreender a definição de interfaces para descrever estruturas de objetos;
- Saber utilizar os tipos personalizados com TypeScript.

### 7.1 Definição de interfaces para descrever a estrutura de objetos

Em TypeScript, assim como em muitas outras linguagens de programação, uma interface é uma estrutura que define um contrato para objetos. Ela descreve os tipos de propriedades ou métodos que um objeto deve ter para ser considerado compatível com essa interface. As interfaces são usadas, principalmente, para garantir a consistência e a integridade do código, tornando-o mais legível e menos propenso a erros.

Vamos deixar mais claro? Pense que o objeto é uma pessoa e a interface seriam as características dessa pessoa, como idade, altura, gênero. Assim, a interface vai definir o objeto a partir de suas características. Utilizar interface para a estrutura de objetos é uma forma de definir a forma ou a estrutura que um objeto deve ter. Isso inclui os tipos de propriedades e métodos que o objeto deve conter. As interfaces são úteis para criar contratos explícitos entre partes do código, permitindo que você descreva as expectativas sobre como os objetos devem ser estruturados e como devem se comportar.

Vamos ver um exemplo de uma estrutura genérica:

```

1 interface MinhaInterface {
2 propriedade1: tipo1;
3 propriedade2: tipo2;
4 metodo1(parametro: tipo): tipoDeRetorno;
5 // Outras propriedades ou métodos podem ser definidos aqui
6 }
7

```

O que podemos observar neste exemplo de definição.

- **MinhaInterface:** local onde definimos o nome da interface;
- **propriedade1** e **propriedade2:** são propriedades (características) que o objeto deve ter, cada propriedade tem um tipo específico (tipo1 e tipo2, respectivamente).
- **metodo1:** definição da ação que o objeto deve ter, que recebe um parâmetro de um determinado tipo (tipo) e retorna um valor de um determinado tipo (tipoDeRetorno).

Vamos visualizar um exemplo na prática:

```

TS Pessoas.ts > ...
1 interface Pessoas {
2 nome: string;
3 idade: number;
4 mensagem(): void;
5 }
6
7 let usuarios: Pessoas = {
8 nome: "João",
9 idade: 30,
10 mensagem() {
11 console.log("Olá, eu sou o João!");
12 }
13 };
14
15 console.log(usuarios.nome); // Saída: João
16 usuarios.mensagem(); // Saída: Olá, eu sou o João!
17

```

Vamos analisar este exemplo prático?

Neste exemplo, podemos observar a definição da interface **Pessoas**, que define a estrutura de um objeto contendo as propriedades: **nome** e **idade**, do tipo *string* e *number*,

respectivamente, e um método (ação), denominado **mensagem**, que não recebe parâmetros e não possui retorno. Em seguida, é declarado um novo objeto denominado **usuários**, sendo este objeto declarado conforme a interface criada, ou seja, é recebido todas as propriedades e métodos definidos na interface.

Como podemos observar, a utilização de interfaces em TypeScript é útil para tornar o código mais legível, auto-documentado e menos propenso a erros, especialmente em situações em que várias partes do código precisam cooperar e interagir entre si de forma consistente.

Para saber mais sobre interfaces em TypeScript, acesse o vídeo explicativo.



Vídeo: *Interfaces - Mini-curso de TypeScript.*

Fonte: [https://www.youtube.com/watch?v=x-q7kGA\\_yp0](https://www.youtube.com/watch?v=x-q7kGA_yp0)

## 7.2 Interfaces personalizadas

As interfaces são usadas para definir tipos personalizados, permitindo a criação de abstrações mais robustas e legíveis no código. Na prática, tal ação melhora o entendimento do código, fornece informações sobre a estrutura dos objetos e permite a verificação estática de tipos.

Vamos visualizar o exemplo simples de definição de interface?

No projeto atual, defina um arquivo denominado **Livros.ts** para implementar o código. Vamos iniciar definindo a interface **Livro** que descreve a estrutura do objeto, apresentando as informações básicas de um livro:

```

TS Livros.ts > ...
1 // Definindo uma interface para representar um livro
2 interface Livro {
3 titulo: string;
4 autor: string;
5 anopublicacao: number;
6 disponivel: boolean;
7 }

```

Em seguida, vamos desenvolver a função que irá utilizar a interface criada.

```

9 // Função que utiliza a interface Livro
10 function infoLivros(livro: Livro): void {
11 console.log(`Título: ${livro.titulo}`);
12 console.log(`Autor: ${livro.autor}`);
13 console.log(`Ano de Publicação: ${livro.anopublicacao}`);
14 console.log(`Disponível: ${livro.disponivel ? 'Yes' : 'No'}`);
15 }

```

A função **infoLivros** recebe um parâmetro do tipo **Livro**, sendo a função utilizada para impressão das informações.

Agora, vamos criar o objeto **MeuLivro**, que deve seguir a estrutura definida pela interface **Livro**. Em seguida, acionamos a função recém-criada passando as informações para o cadastro de um livro.

```

17 // Utilizando a função com um objeto que segue a interface Livro
18 const meuLivro: Livro = {
19 titulo: 'Dom Casmurro',
20 autor: 'Machado de Assis',
21 anopublicacao: 1899,
22 disponivel: true,
23 };

```

Por fim, acionamos a função para imprimir as informações adicionadas.

```

25 // Chamando a função para imprimir informações sobre o livro
26 infoLivros(meuLivro);
27

```

Veja o código completo:

```

TS Livros.ts > ...
1 // Definindo uma interface para representar um livro
2 interface Livro {
3 titulo: string;
4 autor: string;
5 anopublicacao: number;
6 disponivel: boolean;
7 }
8
9 // Função que utiliza a interface Livro
10 function infoLivros(livro: Livro): void {
11 console.log(`Título: ${livro.titulo}`);
12 console.log(`Autor: ${livro.autor}`);
13 console.log(`Ano de Publicação: ${livro.anopublicacao}`);
14 console.log(`Disponível: ${livro.disponivel ? 'Yes' : 'No'}`);
15 }
16
17 // Utilizando a função com um objeto que segue a interface Livro
18 const meuLivro: Livro = {
19 titulo: 'Dom Casmurro',
20 autor: 'Machado de Assis',
21 anopublicacao: 1899,
22 disponivel: true,
23 };
24
25 // Chamando a função para imprimir informações sobre o livro
26 infoLivros(meuLivro);
27

```

Ao utilizar interfaces, você pode garantir que objetos em seu código sigam um contrato específico, tornando o código mais seguro e mais fácil de entender. Isso também facilita a manutenção e a colaboração entre membros da equipe, pois as interfaces servem como documentação sobre a estrutura esperada dos objetos.

Vamos praticar um pouco mais sobre interfaces?

## **DESAFIO PRÁTICO**

### Utilização de interfaces personalizadas.

---

#### **Descrição**

Você trabalha para a empresa "Tudo na Web" como especialista no desenvolvimento de soluções práticas para o dia a dia de estudantes, e recebeu a solicitação para a criação de uma página para agendamento de tarefas, a qual toda a lógica deve ser desenvolvida utilizando *TypeScript*. A página deverá ter as seguintes informações: número da tarefa (id), descrição e *status* da tarefa, ou seja, se foi ou não completada. Trabalhando em conjunto com seus colegas, a meta é desenvolver uma estrutura do código contendo a interface para cadastrar as informações necessárias de uma tarefa. Em seguida, complete o código com uma função utilizada para preencher as informações da tarefa e, por fim, adicione informações para status e as exiba.

---

#### **Objetivos**

Implementação de interfaces personalizadas:

- definição de interfaces para descrever estruturas de objetos;
  - uso de tipos personalizados com Typescript;
  - utilização de interfaces personalizadas.
- 

### Orientações

- Desafiar os participantes a utilizar análise dos dados para criação de uma interface personalizada;
- Estimular o uso eficiente de variáveis, evitando redundâncias e mantendo a coerência no código.



## 7.2.1 Aplicação de extends

Em TypeScript, as interfaces também podem estender outras interfaces e, para isso, utilizamos a palavra reservada **extends** para compor tipos mais complexos a partir de tipos simples. A partir do uso da palavra *extends*, a nova interface criada passa a ter todos os elementos declarados mais os elementos da interface de extensão.

Seguindo o exemplo de que objeto é uma pessoa e as interfaces suas características, seria como se eu definisse a pessoa, por exemplo: uma mulher de 38 anos com altura de 1.60 e o seu nome é Maria. Então, eu seleciono Maria e vou adicionar uma roupa nela. Isso seria a função do *extend*, pegar uma pessoa já definida e atribuir mais algumas coisas a ela. Vamos ver um pouco mais sobre isso?

Para compreender melhor, vamos alterar o exemplo apresentado anteriormente sobre cadastro de livros, mudando o arquivo contendo a interface **Livro**. Imaginemos a seguinte situação: quando falamos sobre livros, já ligamos a informação que um livro pertence a uma biblioteca, onde não temos apenas livros, mas também, outros itens, como revistas, mídias, entre outros. Neste caso, podemos criar uma interface **ItensBiblioteca**.

```
TS Livros.ts > ...
1 // Definindo uma interface base para itens de biblioteca
2 interface ItemBiblioteca {
3 id: number;
4 titulo: string;
5 disponivel: boolean;
6 }
7
```

A partir da criação da interface ItemBiblioteca, a interface Livro pode ser alterada, passando a ter suas propriedades e receber como extensão as propriedades da interface recém-criada.



```

8 // Definindo uma interface para representar informações sobre um livro
9 interface Livro extends ItemBiblioteca {
10 autor: string;
11 anopublicacao: number;
12 }

```

A partir do uso da palavra reservada **extends**, a interface **Livro** passa a ter além das propriedades declaradas em seu corpo, todas as propriedades da interface **ItemBiblioteca**. É possível visualizar as propriedades no momento da declaração da função.

```

14 // Função que utiliza a interface Livro
15 function infoLivros(livro: Livro): void {
16 console.log(`Id do Livro: ${livro.id}`);
17 console.log(`Título: ${livro.titulo}`);
18 console.log(`Autor: ${livro.autor}`);
19 console.log(`Ano de Publicação: ${livro.anopublicacao}`);
20 console.log(`Disponível: ${livro.disponivel ? 'Yes' : 'No'}`);
21 }

```

Por fim, para testar os resultados, vamos utilizar a função e acioná-la para impressão no console do resultado.

```

23 // Utilizando a função com um objeto que segue a interface Livro
24 const meuLivro: Livro = {
25 id: 1,
26 titulo: 'Dom Casmurro',
27 autor: 'Machado de Assis',
28 anopublicacao: 1899,
29 disponivel: true,
30 };
31
32 // Chamando a função para imprimir informações sobre o livro
33 infoLivros(meuLivro);
34

```

```

TS Livros.ts > ...
1 // Definindo uma interface base para itens de biblioteca
2 interface ItemBiblioteca {
3 id: number;
4 titulo: string;
5 disponivel: boolean;
6 }
7
8 // Definindo uma interface para representar informações sobre um livro
9 interface Livro extends ItemBiblioteca {
10 autor: string;
11 anopublicacao: number;
12 }
13
14 // Função que utiliza a interface Livro
15 function infoLivros(livro: Livro): void {
16 console.log(`Id do Livro: ${livro.id}`);
17 console.log(`Título: ${livro.titulo}`);
18 console.log(`Autor: ${livro.autor}`);
19 console.log(`Ano de Publicação: ${livro.anopublicacao}`);
20 console.log(`Disponível: ${livro.disponivel ? 'Yes' : 'No'}`);
21 }
22
23 // Utilizando a função com um objeto que segue a interface Livro
24 const meuLivro: Livro = {
25 id: 1,
26 titulo: 'Dom Casmurro',
27 autor: 'Machado de Assis',
28 anopublicacao: 1899,
29 disponivel: true,
30 };
31
32 // Chamando a função para imprimir informações sobre o livro
33 infoLivros(meuLivro);
34

```

## 7.2.2 Utilizando tipos

Além de interfaces, você pode usar a palavra-chave **type** para criar tipos personalizados. Tipos são frequentemente usados para definir tipos de união, interseção e outros mais complexos.

Vamos a um exemplo prático referente a tipos:

Adicione um arquivo denominado **coordenadas.ts** em seu projeto. Em seguida, vamos iniciar a codificação definindo o tipo:

```

TS Coordenadas.ts > ...
1 // Definindo um tipo denominado Coordenada
2 type Coordenada = {
3 x: number;
4 y: number;
5 };
6

```

Vamos utilizar o tipo criado em uma função:

```

7 // Usando o tipo 'Coordenadas em uma função
8 function infoCoordenada(coord: Coordenada) {
9 console.log(`Coordenadas: (${coord.x}, ${coord.y})`);
10 }
11

```

Por fim, vamos visualizar a função:

```

12 const ponto: Coordenada = { x: 10, y: 20 };
13 infoCoordenada(ponto);
14

```

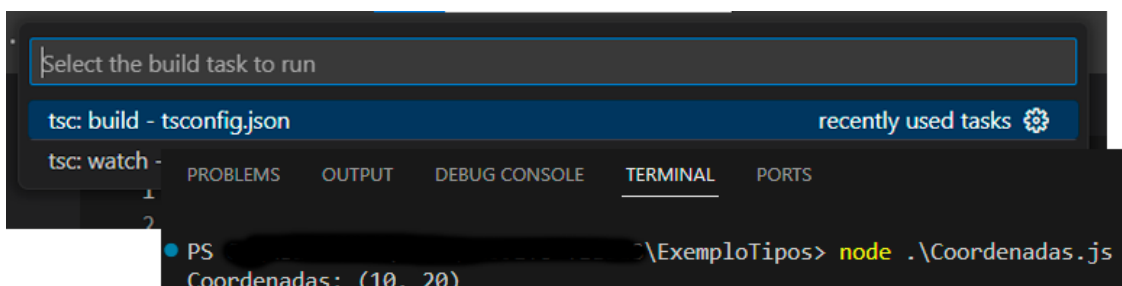
Veja o código completo:

```

TS Coordenadas.ts > ...
1 // Definindo um tipo denominado Coordenada
2 type Coordenada = {
3 x: number;
4 y: number;
5 };
6
7 // Usando o tipo 'Coordenadas' em uma função
8 function infoCoordenada(coord: Coordenada) {
9 console.log(`Coordenadas: (${coord.x}, ${coord.y})`);
10 }
11
12 const ponto: Coordenada = { x: 10, y: 20 };
13 infoCoordenada(ponto);
14

```

Agora, basta transpilar o código para visualizar o resultado.



Quer saber mais sobre interfaces e tipos personalizados? Acesse o vídeo.



Vídeo *Curso TypeScript - Aula 9: Extendendo interfaces e tipos*, do canal Daniel Berg. Disponível em: <https://www.youtube.com/watch?v=WkPJnz3uUhA>. Acesso em: 13 mar. 2024.

Agora que conhecemos o conceito de interfaces e tipos em TypeScript, vamos debater com os colegas sobre o assunto?



## DESAFIO PRÁTICO

### Definição de interfaces e tipos personalizados.

---



#### Descrição

Após visualizarmos os conceitos de interfaces e tipos personalizados, vamos debater com os colegas sobre o assunto.

Você sabia que a área Playground disponível no *site* oficial do TypeScript além de ser uma área para edição e transpilar código oferece diversos exemplos de codificação para analisarmos?

Colocando essa análise em prática, vamos visualizar o exemplo apresentado sobre a criação e manipulação de interfaces e tipos disponível em: [exemploInterfaceseTipos](#) e debater com os colegas os seguintes elementos:

- quantas interfaces foram declaradas no código apresentado?
  - quantas tipos foram declaradas no código apresentado?
  - foi aplicado o recurso *extends*? Caso não, como o recurso poderia ser utilizado no exemplo apresentado?
- 



#### Objetivos

Implementação de interfaces personalizadas:

- definição de interfaces para descrever estruturas de objetos;
  - uso de tipos personalizados com Typescript;
  - utilização de interfaces personalizadas.
- 

#### Orientações

- Desafiar os participantes a analisar códigos já implementados, oferecendo uma visão pessoal sobre o conteúdo;

- Estimular o uso eficiente dos recursos de interfaces e tipos, evitando redundâncias e mantendo a coerência no código.



Em TypeScript, interfaces e tipos personalizados são maneiras de definir a forma de objetos. Permite a criação de tipos a partir dos existentes usando uma grande variedade de operadores. Além disso, os tipos **union** (união) e **intersection** (interseção) são recursos poderosos que permitem combinar ou interseccionar tipos.

Vamos explorar esses conceitos?

## 7.3 Tipos union

Os tipos **union** (união) são uma forma de **indicar que um determinado valor pode ter mais de um tipo**, utilizando o **operador barra vertical** - "|". Esse recurso é muito útil quando uma função ou variável pode aceitar mais de um tipo.

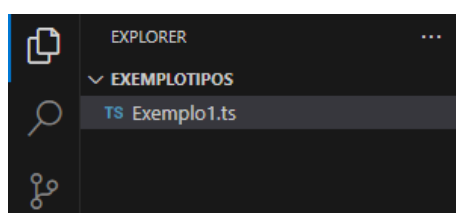
Os tipos **union** são úteis quando se quer fornecer uma anotação de tipo explícita para uma variável, mesmo que ela já possua um valor inicial. Por exemplo, na questão dos livros, você tem o ano de publicação do livro (valor inicial) e quer adicionar a ele uma outra informação, como o nome, então você optaria por usar o **union**, que vai unir os dois valores.

### 7.3.1 Definindo um tipo de união

Vamos entender um pouco mais?

Vamos iniciar um projeto para apresentar os conceitos dos tipos e, para tal, **crie uma pasta no diretório raiz** denominada **"ExemploTipos"**. Nessa pasta vamos criar códigos para exemplificar os tipos em TypeScript.

Após criação da pasta vamos iniciar nosso primeiro exemplo, crie **um arquivo** denominado **"Exemplo1.ts"**.



Em seguida, vamos criar e configurar o arquivo **'tsconfig.json'** utilizado para transpilar os arquivos *TypeScript*.

```
PS \ExemploTipos> tsc --init

Created a new tsconfig.json with:

 target: es2016
 module: commonjs
 strict: true
 esModuleInterop: true
 skipLibCheck: true
 forceConsistentCasingInFileNames: true

You can learn more at https://aka.ms/tsconfig
PS \ExemploTipos>
```

Agora, com um novo ambiente de projeto configurado, partimos para **editar o arquivo "Exemplo1.ts"**, onde o objetivo é apresentar o primeiro exemplo da utilização do tipo *union*. Neste primeiro exemplo, vamos utilizar um recurso simples de um tipo denominado **NumeroOuString**, que poderá receber valores numéricos ou texto.

No arquivo Exemplo1.ts, declare o tipo da seguinte forma:

```
type NumeroOuString = number | string;
```

Na linha de código acima, a declaração do tipo **NumeroOuString** apresenta união entre os tipos de dados **number** e **string**. O símbolo barra vertical "|" é utilizado para indicar a utilização de união. Após a declaração do tipo, vamos desenvolver uma função para exibir no console os valores recebidos.

```
function imprimirValor(valor: NumeroOuString): void {
 console.log(valor);
}
```

Logo após a declaração da função, vamos acioná-la passando valores por parâmetro.

```
imprimirValor(42);
imprimirValor("Olá");
```

Vamos visualizar o código completo:

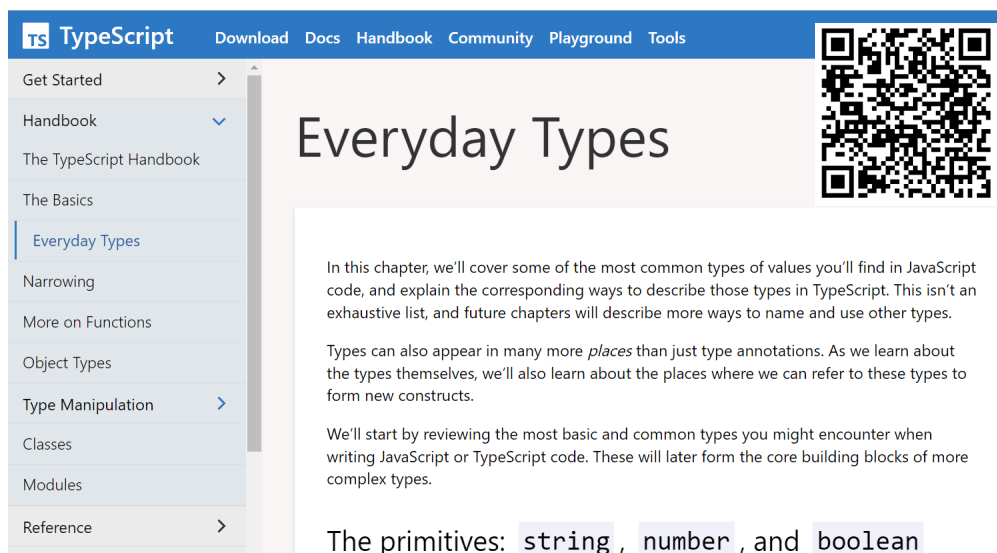
```
TS Exemplo1.ts > ...
1 type NumeroOuString = number | string;
2
3 function imprimirValor(valor: NumeroOuString): void {
4 | console.log(valor);
5 }
6 imprimirValor(42);
7 imprimirValor("Olá");
8
9
```

Segundo Goldberg (2022, p. 42) “Uniãos – Expansão do tipo permitido de um valor para que use dois ou mais tipos possíveis”.

Como podemos visualizar no primeiro exemplo, o tipo **NumeroOuString** pode receber tanto valores numéricos quanto valores de texto. Agora, basta transpilar o código para visualizar o resultado. Após o código transpilado, o mesmo tipo pode receber, em momentos diferentes, dados numéricos e textos.

## Saiba mais!

Para saber mais sobre os tipos union acesse o site oficial do TypeScript.

The image is a screenshot of the TypeScript Handbook website. The top navigation bar includes links for 'Download', 'Docs', 'Handbook', 'Community', 'Playground', and 'Tools'. A sidebar on the left lists various sections: 'Get Started', 'Handbook' (expanded), 'The TypeScript Handbook', 'The Basics', 'Everyday Types' (highlighted), 'Narrowing', 'More on Functions', 'Object Types', 'Type Manipulation', 'Classes', 'Modules', and 'Reference'. The main content area is titled 'Everyday Types' and contains introductory text about common types in JavaScript and TypeScript. It mentions that the chapter will cover basic types and how they are used in code. At the bottom of the visible text, it lists the primitives: 'string', 'number', and 'boolean'.

Fonte: <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#union-types>.

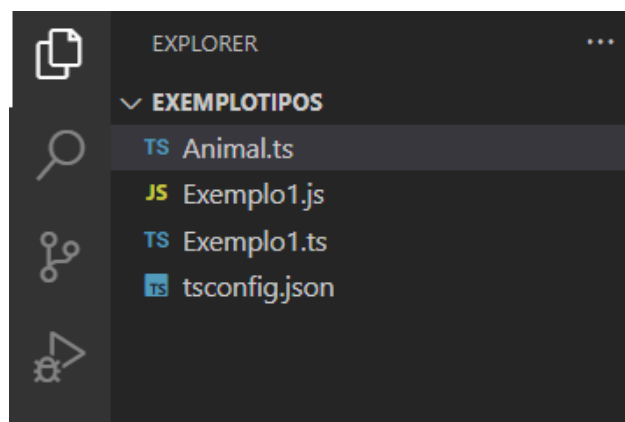




## Na prática!

Para visualizarmos na prática a utilização do tipo **union** para criação de interfaces, vamos **modelar classes de diferentes tipos de animais em um sistema**. Alguns animais podem voar enquanto outros nadam. Vamos criar uma interface **Animal**, que utiliza *union type* para representar essa situação.

Para iniciar o desenvolvimento, vamos utilizar o mesmo projeto já criado anteriormente, adicionando apenas um novo arquivo TypeScript. Crie um novo arquivo que será nossa interface denominado **Animal.ts**.



Vamos iniciar a codificação definindo um tipo de união para características específicas do animal.

```
type Caracteristicas = {voa: boolean} | {nada: boolean};
```

Na linha de código acima, a propriedade **“Caracteristicas”** recebe os valores **“voa”** e **“nada”**, ambos do tipo **boolean**.

Agora, vamos definir a interface **“Animal”**, que contém duas variáveis, **“nome”** do tipo **string** e **“características”** do tipo **“Caracteristicas”**, propriedade definida anteriormente.

```
interface Animal {
 nome: string;
 caracteristicas: Caracteristicas;
}
```

Para verificar, vamos adicionar exemplos de animais:

```
const passaro: Animal = {nome: "Pássaro", caracteristicas: {voa: true} };
const peixe: Animal = {nome: "Peixe", caracteristicas: {nada: true} };
```

A função **VerificarCaracteristicas** será utilizada para verificar as características do animal inserido.

```
function verificarCaracteristicas(animal: Animal): void {
 console.log(`Nome: ${animal.nome}`);

 if ('voa' in animal.caracteristicas) {
 console.log("Pode voar");
 }

 if ('nada' in animal.caracteristicas) {
 console.log("Pode nadar");
 }
}
```

Por fim, vamos testar a função com os exemplos.

```
verificarCaracteristicas(passaro);
verificarCaracteristicas(peixe);
```

Visualize o código completo:

```

TS Animal.ts > ...
1 // Definindo um tipo de união para características específicas do animal
2 type Caracteristicas = { voa: boolean } | { nada: boolean };
3 // Interface para o Animal
4 interface Animal {
5 nome: string;
6 caracteristicas: Caracteristicas;
7 }
8 // Exemplos de animais
9 const passaro: Animal = { nome: "Pássaro", caracteristicas: { voa: true } };
10 const peixe: Animal = { nome: "Peixe", caracteristicas: { nada: true } };
11 // Função que verifica as características do animal
12 function verificarCaracteristicas(animal: Animal): void {
13 console.log(`Nome: ${animal.nome}`);
14
15 if ('voa' in animal.caracteristicas) {
16 console.log("Pode voar");
17 }
18
19 if ('nada' in animal.caracteristicas) {
20 console.log("Pode nadar");
21 }
22 }
23 // Testando a função com os exemplos
24 verificarCaracteristicas(passaro); // Pode voar
25 verificarCaracteristicas(peixe); // Pode nadar
26

```

No exemplo apresentado, a interface **Animal** possui uma propriedade denominada **"caracteristicas"**, que é **uma união de tipos, representando a capacidade de voar ou nadar**. A função **verificarCaracteristicas** utiliza a estrutura de controle **if ('voa' in animal.caracteristicas)** para **verificar se o animal pode voar** e **if ('nada' in animal.caracteristicas)** para **verificar se o animal pode nadar**. Agora, basta transpilar o código para visualizar o resultado.

Trabalhar com propriedades do tipo *union* permite a criação de interfaces mais flexíveis para representar diferentes características ou comportamentos dentro de um mesmo tipo, tornando seu código mais modular e adaptável.

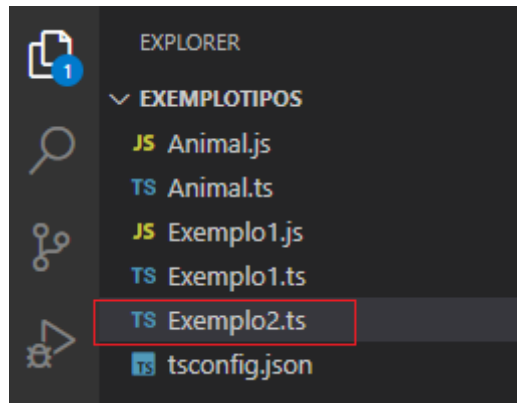
## 7.4 Tipos *intersection*

Em TypeScript, o tipo ***intersection*** é utilizado para **combinar dois ou mais tipos, criando um novo que contém todas as propriedades de cada tipo individual**. Assim, como no tipo *union*, para a criação do tipo *intersection*, é utilizado um operador, neste caso, o operador utilizado é o **"&"**.

Ao se utilizar o *intersection*, a interface resultante representará um novo tipo que possui todas as propriedades de cada tipo individual.

Vamos ver um exemplo?

Vamos continuar utilizando o projeto criado anteriormente, apenas inserindo um novo arquivo denominado "Exemplo2.ts".



Fonte: elaborado pelo autor (2024).

O objetivo é a definição de duas interfaces, **Pessoa e Endereco**, e a criação de uma terceira interface denominada **PessoaComEndereco**, que combina as propriedades das duas interfaces anteriores.

No arquivo Exemplo2.ts, iniciamos declarando a interface **Pessoa** da seguinte forma:

```
type Pessoa = {
 nome: string;
 idade: number;
};
```

Em seguida, vamos adicionar as propriedades da interface Endereco:

```
type Endereco = {
 rua: string;
 cidade: string;
};
```

Ao criar a interface PessoaComEndereco, adicionamos as propriedades da interface Endereco e Pessoa e, para isso, utilizamos o símbolo & para adicionar as propriedades:

```
type PessoaComEndereco = Pessoa & Endereco;
```

Vamos exemplificar a utilização:

```
const pessoaComEndereco: PessoaComEndereco = {
 nome: 'Ana',
 idade: 30,
 rua: 'Rua Principal',
 cidade: 'Cidade B',
};
```

Por fim, vamos exibir o resultado:

```
console.log(pessoaComEndereco);
```

Visualize o código completo:

```
TS Exemplo2.ts > ...
1 type Pessoa = {
2 nome: string;
3 idade: number;
4 };
5
6 type Endereco = {
7 rua: string;
8 cidade: string;
9 };
10
11 // Criando um novo tipo usando '&', que é uma intersection
12 type PessoaComEndereco = Pessoa & Endereco;
13
14
15 const pessoaComEndereco: PessoaComEndereco = {
16 nome: 'Ana',
17 idade: 30,
18 rua: 'Rua Principal',
19 cidade: 'Cidade B',
20 };
21
22 console.log(pessoaComEndereco);
23
```

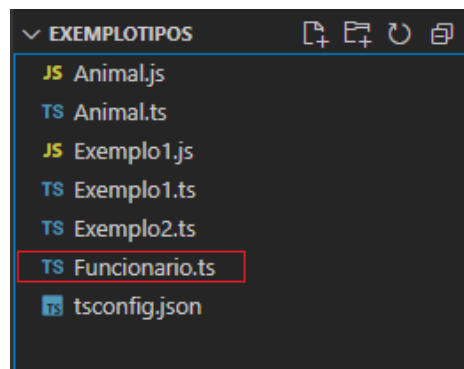
No exemplo apresentado, a interface `PessoaComEndereco` é um novo tipo que combina as propriedades das interfaces `Pessoa` e `Endereco` usando o operador `&`. Assim, um objeto do tipo `PessoaComEndereco` deve ter tanto as propriedades de `Pessoa` quanto as de `Endereco`.

## Na prática!

Vamos criar um exemplo prático para entender como a interseção (&) pode ser utilizada em TypeScript. Iremos exemplificar a construção de um sistema de gerenciamento de funcionários:

Você tem diferentes tipos de funcionários, alguns são gerentes e outros, desenvolvedores. Vamos modelar isso usando a interseção de tipos.

Para iniciar o desenvolvimento, vamos utilizar o mesmo projeto criado anteriormente, adicionando apenas um novo arquivo TypeScript. Crie um novo arquivo, que será nossa interface denominado **Funcionarios.ts**.



Vamos iniciar a codificação definindo um tipo para informações básicas do cadastro de uma Pessoa contendo três dados: **nome, idade e CPF**.

```
type CadPessoa = {
 nome: string;
 idade: number;
 cpf: number;
};
```

Em seguida, vamos definir um tipo para informações específicas de um funcionário, onde será inserido **cargo e salário**.

```
type Funcionario = {
 cargo: string;
 salario: number;
};
```

Após a definição das interfaces com informações específicas, vamos adicionar a **interseção** de tipos para representar uma **Pessoa Trabalhadora**, que envolve informações de **CadPessoa** e **Funcionario**.

```
type PessoaTrabalhadora = CadPessoa & Funcionario;
```

Vamos exemplificar a utilização:

```
const funcionario: PessoaTrabalhadora = {
 nome: "João",
 idade: 25,
 cpf: 12314589011,
 cargo: "Desenvolvedor",
 salario: 50000,
};
```

E exibir o resultado no console:

```
console.log(funcionario);
```

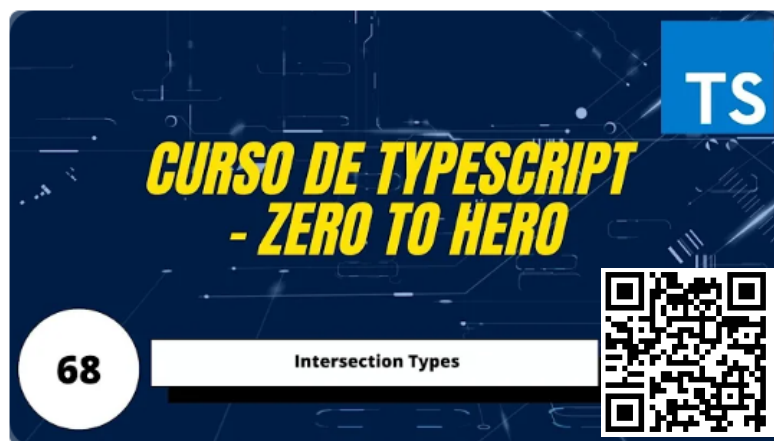
Veja a codificação completa:

```
1 type CadPessoa = {
2 nome: string;
3 idade: number;
4 cpf: number;
5 };
6
7 type Funcionario = {
8 cargo: string;
9 salario: number;
10 };
11
12 type PessoaTrabalhadora = CadPessoa & Funcionario;
13
14 const funcionario: PessoaTrabalhadora = {
15 nome: "João",
16 idade: 25,
17 cpf: 12314589011,
18 cargo: "Desenvolvedor",
19 salario: 50000,
20 };
21
```

No exemplo apresentado, **PessoaTrabalhadora** é um tipo que contém todas as propriedades de **CadPessoa** e **Funcionario**. Portanto, um objeto do tipo **PessoaTrabalhadora** contém tanto as propriedades de **CadPessoa** quanto as de **Trabalhador**.

## Saiba mais!

Para saber mais sobre os tipos *union* e *intersection*, acesse o vídeo.



Vídeo *TypeScript - Zero to Hero | Intersection Types*, do canal Gláucia Lemos. Disponível em:

<https://youtu.be/2hE57wR86YY>. Acesso em: 13 mar. 2024.

## 7.5 Combinação de tipos com *union* e *intersection*

Em TypeScript, a combinação de tipos é uma prática fundamental para criar estruturas de dados flexíveis e expressivas. Duas maneiras principais de combinar tipos são através da união (*union*) e da interseção (*intersection*).

Conforme já visto, o ***union types*** permite que uma variável, parâmetro ou propriedade possa ter mais de um tipo. Isso é feito utilizando o **operador “|”**. Enquanto o tipo ***intersection types*** permite combinar múltiplos tipos criando um novo tipo que possui todas as propriedades de cada tipo individual. Isso é feito utilizando o **operador “&”**. A utilização de *union* e *intersection* juntos pode criar tipos complexos e flexíveis.



### Na prática!

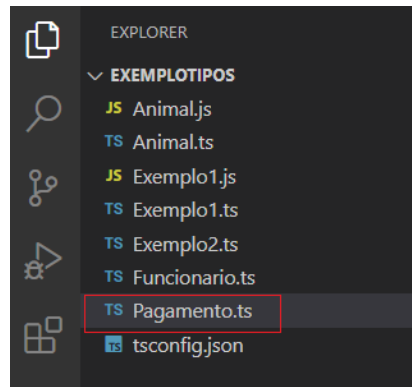
Vamos colocar em prática a combinação de *union* e *intersection*?

Suponha que você esteja modelando um sistema de pagamento para uma loja,



no qual os usuários podem realizar transações de diferentes tipos, como transferências bancárias ou pagamentos com cartão de crédito. Além disso, você deseja registrar se a transação foi bem-sucedida ou não. Vamos utilizar a combinação dos tipos union e intersection para modelar essas situações.

Para iniciar, crie o arquivo **Pagamento.ts** na pasta de trabalho:



Logo após, vamos inserir a codificação correspondente a interface **InfoUsuario** que será utilizada para definir as informações básicas do usuário do sistema de pagamento.

```
interface InfoUsuario {
 nome: string;
 saldo: number;
}
```

A interface **InfoTransação** será utilizada para definir as informações básicas sobre as transações de pagamento.

```
interface InfoTransacao {
 valor: number;
 data: Date;
}
```

A interface **InfoTransferencia** define as informações específicas de uma transferência bancária no caso da escolha desta opção de pagamento.

```
interface InfoTransferencia {
 tipo: "transferencia";
 contaDestino: string;
}
```

A interface **InfoPagamentoCartao** define as informações específicas para pagamento com cartão de crédito.

```
interface InfoPagamentoCartao {
 tipo: "cartao";
 numeroCartao: string;
}
```

Agora, vamos criar duas interfaces para verificar se a transação de pagamento foi ou não bem-sucedida.

```
// Definindo um tipo para informações específicas de uma transação bem-sucedida
interface InfoTransacaoBemSucedida {
 sucesso: true;
}

// Definindo um tipo para informações específicas de uma transação mal-sucedida
interface InfoTransacaoMalSucedida {
 sucesso: false;
 motivo: string;
}
```

Definida as interfaces personalizadas, vamos criar a propriedade **Transação**, que irá utilizar o **union type** para representar uma transação que pode ser de transferência ou cartão bem-sucedida ou malsucedida.

```
type Transacao = InfoTransacao & (InfoTransferencia | InfoPagamentoCartao) &
(InfoTransacaoBemSucedida | InfoTransacaoMalSucedida);
```

Agora vamos exemplificar a utilização:

```
const usuario: InfoUsuario = { nome: "Alice", saldo: 1000 };

const transferenciaBemSucedida: Transacao = {
 valor: 200,
 data: new Date(),
 tipo: "transferencia",
 contaDestino: "98765432",
 sucesso: true,
};

const pagamentoCartaoMalSucedido: Transacao = {
 valor: 50,
```

```

data: new Date(),
tipo: "cartao",
numeroCartao: "**** * 1234",
sucesso: false,
motivo: "Saldo insuficiente",
};

```

Neste exemplo, **Transacao** é uma combinação de **InfoTransacao** com a união de **InfoTransferencia** e **InfoPagamentoCartao**, e também uma união de **InfoTransacaoBemSucedida** e **InfoTransacaoMalSucedida**. Isso permite modelar transações de diferentes tipos, bem como seu resultado (bem-sucedida ou malsucedida).

Os exemplos de **transferenciaBemSucedida** e **pagamentoCartaoMalSucedido** ilustram como você pode criar instâncias de **Transacao** com diferentes conjuntos de propriedades, dependendo do tipo e do resultado da transação.

Veja a codificação completa:

```

TS Pagamento.ts > ...
1 // Definindo um tipo para informações básicas do usuário
2 interface InfoUsuario {
3 nome: string;
4 saldo: number;
5 }
6
7 // Definindo um tipo para informações específicas da transação
8 interface InfoTransacao {
9 valor: number;
10 data: Date;
11 }
12
13 // Definindo um tipo para informações específicas de uma transferência bancária
14 interface InfoTransferencia {
15 tipo: "transferencia";
16 contaDestino: string;
17 }
18
19 // Definindo um tipo para informações específicas de um pagamento com cartão de crédito
20 interface InfoPagamentoCartao {
21 tipo: "cartao";
22 numeroCartao: string;
23 }
24
25 // Definindo um tipo para informações específicas de uma transação bem-sucedida
26 interface InfoTransacaoBemSucedida {
27 sucesso: true;
28 }

```

```

28 }
29
30 // Definindo um tipo para informações específicas de uma transação mal-sucedida
31 interface InfoTransacaoMalSucedida {
32 sucesso: false;
33 motivo: string;
34 }
35
36 // Union type para representar uma transação que pode ser de transferência ou cartão, bem-sucedida ou mal-sucedida
37 type Transacao = InfoTransacao & (InfoTransferencia | InfoPagamentoCartao) &
38 (InfoTransacaoBemSucedida | InfoTransacaoMalSucedida);
39
40 // Exemplos de uso
41 const usuario: InfoUsuario = { nome: "Alice", saldo: 1000 };
42
43 const transferenciaBemSucedida: Transacao = {
44 valor: 200,
45 data: new Date(),
46 tipo: "transferencia",
47 contaDestino: "98765432",
48 sucesso: true,
49 };

```

```

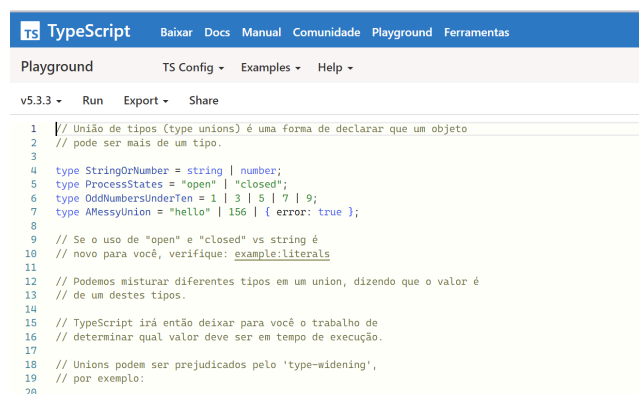
50
51 const pagamentoCartaoMalSucedido: Transacao = {
52 valor: 50,
53 data: new Date(),
54 tipo: "cartao",
55 numeroCartao: "**** * 1234",
56 sucesso: false,
57 motivo: "Saldo insuficiente",
58 };
59

```

Entender e aplicar a combinação de tipos com *union* e *intersection* é fundamental para criar código TypeScript robusto e adaptável, permitindo uma modelagem mais precisa e expressiva das estruturas de dados em sua aplicação.

## Saiba mais!

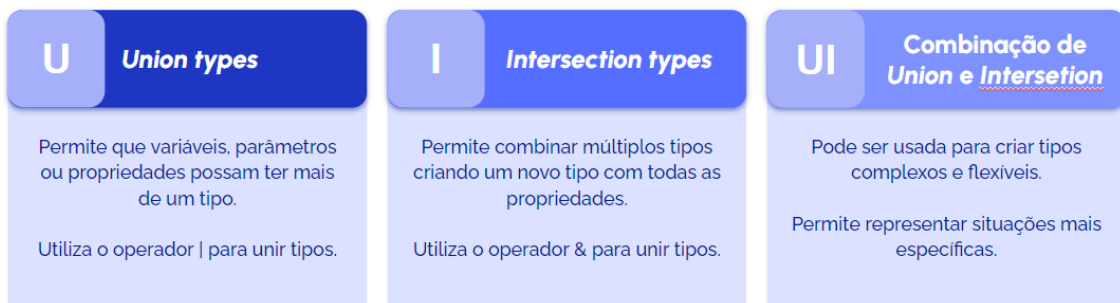
Para obter mais exemplos contendo a combinação dos tipos *union* e *intersection*, acesse o site oficial do TypeScript.



Fonte: <https://www.typescriptlang.org/pt/play#example/union-and-intersection-types>.

A combinação de interfaces, tipos personalizados, tipos de união e tipos de interseção oferece uma grande flexibilidade para criar estruturas de dados complexas e expressivas em TypeScript. Esses recursos são fundamentais para tornar o código mais compreensível, seguro e fácil de manter.

## Tipos personalizados TypeScript



Tipos personalizados permitem dar nomes a tipos existentes ou criar novos tipos compostos, proporcionando clareza e expressividade ao código. Com auxílio das combinações *union* e *intersection types*, o código torna-se mais legível, seguro e flexível. Eles promovem a expressividade e a consistência, facilitando o desenvolvimento de sistemas robustos e de fácil manutenção.

Ao entender e aplicar esses conceitos, os desenvolvedores podem criar estruturas de dados complexas e expressivas de maneira eficiente.





## ATIVIDADE DE FIXAÇÃO

1. Explique o conceito de interfaces em TypeScript. Como as interfaces contribuem para a criação de contratos e qual é sua principal finalidade em desenvolvimento de *software*.
2. Explique o que são *intersection types* em TypeScript e como eles podem ser úteis na criação de tipos compostos.
3. Descreva o conceito de *union types* em TypeScript. Como você pode usar *union types* para expressar a possibilidade de uma variável ter mais de um tipo?
4. Como a utilização de *union types* e *intersection types* em TypeScript proporciona flexibilidade na definição de tipos de dados?
5. Como as interfaces são aplicadas para definir a estrutura de objetos em TypeScript?
6. Com a aplicação do conceito *intersection*, como você poderia usar uma interseção para representar um valor que precisa ser uma *string* e um número?
7. Qual é o operador usado para definir uma interseção em TypeScript?
8. Quais são as melhores práticas ao nomear interfaces em TypeScript? Como você escolheria um nome significativo para uma interface?
9. Explique o porquê as interfaces são úteis na definição de estruturas de objetos.
10. Como você pode utilizar interfaces para garantir a consistência e padronização nos contratos de funções em uma aplicação TypeScript? Apresente um exemplo prático.

# CAPÍTULO 08

## DECLARAÇÃO DE CLASSES EM TYPESCRIPT

### O que esperar deste capítulo:

- Saber a sintaxe de declaração de classes em TypeScript;
- Entender a manipulação de propriedades e métodos em TypeScript;
- Compreender conceitos avançados como herança, modificadores de acesso e encapsulamento.

Vamos relembrar a importância da **orientação a objetos** no universo da programação?

A **orientação a objetos** é muito importante para **criar softwares**. Ela nos oferece uma forma organizada e eficiente de **criar, planejar e construir sistemas complicados**.



Uma grande vantagem de usar a orientação a objetos é que podemos **criar programas que se parecem mais com o mundo real**. Em vez de lidar com linhas complicadas de código, **podemos representar as coisas que existem no mundo real, como objetos**.

### 8.1 Declaração de classes em TypeScript

No dia a dia, a utilização da orientação a objetos no JavaScript pode gerar algumas dúvidas e dificuldades no momento da execução. Assim, a utilização do TypeScript torna tudo mais simples, contribuindo na compreensão do que está sendo feito, oferecendo a possibilidade de utilizarmos **classes, interfaces, encapsulamento e herança**.

Quando estamos usando a programação orientação a objetos (POO), **criar classes é uma parte super importante**, pois elas nos **permitem representar coisas do mundo**

**real ou ideias abstratas** dentro do nosso programa.

Vamos entender melhor?

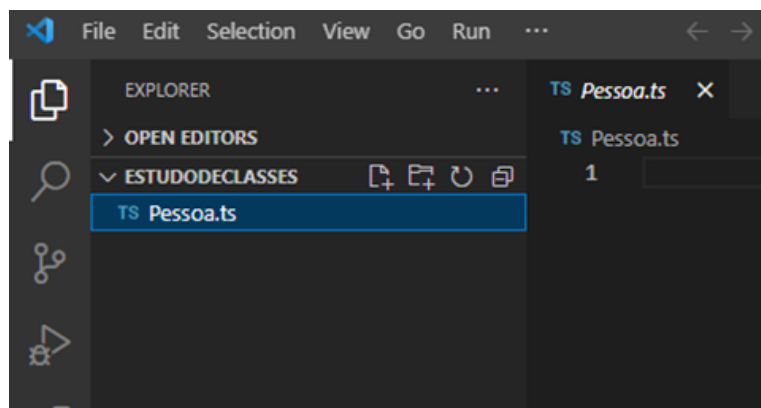
Imagine que foi iniciado o projeto de construção de uma nova casa. Para isso, é necessário a definição de uma planta, o tamanho dos cômodos e onde vão ficar no terreno, o posicionamento dos móveis, e assim por diante. Após a criação da planta, é possível criar diversas casas.

Assim como a planta é utilizada para modelar o projeto de uma casa e gerar vários outros imóveis com as mesmas características, na programação utilizamos **as classes** para modelar um objeto, definindo sua estrutura de dados e seu comportamento, podendo reutilizar esse modelo em outros objetos.

**Uma classe é uma estrutura que encapsula dados (atributos) e comportamentos (métodos).**

No TypeScript, **uma classe é uma estrutura que define o layout de um objeto e serve como um modelo para criar objetos**. Em TypeScript, uma classe é definida usando a palavra-chave **class**.

Para melhor ilustrar a criação de classes no TypeScript vamos iniciar um novo projeto. Para isso, vamos criar um novo diretório denominado **EstudodeClasses** e, em seguida, vamos criar um primeiro arquivo TypeScript denominado **Pessoa**.



Após criação do diretório de trabalho e arquivo TypeScript, vamos iniciar a codificação. Conforme já mencionado anteriormente, utilizamos a palavra reservada **class** para determinar o nome de uma classe.

```
class Pessoa {
```

**Logo após determinar o nome da classe, utilizamos o símbolo chave { para indicar a sua abertura.**



## 8.2 Propriedades de uma classe em TypeScript

Uma classe pode conter **métodos (funções)** e **propriedades (variáveis)** que definem o comportamento e os dados do objeto. Vamos definir as propriedades da classe?

As propriedades são **variáveis que são definidas dentro do escopo da classe** e estão associadas a cada instância (objeto) da classe. Elas representam os dados ou características que pertencem a cada objeto criado naquela classe. As propriedades são definidas fora dos métodos e **podem ter qualquer tipo de dado válido** em TypeScript. Para este exemplo, vamos adicionar duas propriedades:

```
nome: string;
idade: number;
```

**Na classe apresentada temos duas propriedades: nome, do tipo *string*, e idade, do tipo numérico.**

As propriedades podem ter modificadores de acesso como *public*, *private* e *protected*, que são acessadas usando a notação de ponto (objeto.propriedade) a partir de uma instância da classe.

## 8.3 Métodos de uma classes em TypeScript

Um método em TypeScript é uma **função que está ligada a uma classe específica e pode ser chamada em instâncias (objetos)**. Esses métodos são utilizados para representar comportamentos associados aos objetos dessa classe e podem manipular os dados contidos no objeto.

**O método deve ser definido dentro do corpo da classe**, utilizando as `{ }` - (chaves) para abertura e fechamento do mesmo. Eles podem ter qualquer nome válido de função e podem aceitar parâmetros.

Um exemplo é o **método construtor** de uma classe, um método especial que é executado automaticamente quando um objeto da classe é criado. Ele tem a **responsabilidade de inicializar os valores das propriedades do objeto e pode aceitar parâmetros para configurar esses valores** durante a criação do objeto.

Após a definição do nome da classe e suas propriedades, vamos declarar o método construtor.

O método construtor é definido usando a palavra-chave ***constructor*** seguida das

características de cada variável da classe que você está criando. Dentro do corpo do construtor, você pode realizar qualquer inicialização necessária para configurar o estado inicial do objeto, ou seja, você quer que o gênero seja masculino, a idade seja 25 e o nome seja Carlos, e com o método construtor, você pode definir e iniciar o programa da seguinte forma:

```
constructor(nome: string, idade: number, genero: string) {
 this.nome = nome;
 this.idade = idade;
}
```

O que significa a palavra **this**?

Métodos de uma classe podem acessar as propriedades da instância (objeto) da classe usando a palavra-chave **this**. Isso permite que os métodos manipulem os dados do objeto atual.

Assim, quando for necessário instanciar um objeto da classe, vamos passar os dados das propriedades.

Vamos verificar como fica a instância de uma classe.

```
let pessoa = new Pessoa('Paulo', 0);
```

No exemplo apresentado foi definido um objeto denominado **pessoa** que recebe uma nova instância da classe Pessoa.

A palavra reservada **new** é usada para criar uma nova instância de um tipo, seja ele uma classe ou um construtor de função. Quando usada com uma classe, **new** aloca memória para um novo objeto e chama automaticamente o método construtor da classe para inicializar esse objeto.

Vamos visualizar o código criado até aqui:

```
class Pessoa {
 nome: string;
 idade: number;

 constructor(nome: string, idade: number) {
 this.nome = nome;
 this.idade = idade;
 }
}
```

```

 }

 }
 let pessoa = new Pessoa('Paulo', 0);

```

Aqui, vamos inserir um método denominado **saudacao**.

```

 saudacao(): void {
 console.log(`Olá, seu nome é ${this.nome} e sua idade é ${this.idade}
anos.`);
 }

```

É importante definir o tipo de retorno de um método para indicar qual tipo de dado o método irá retornar quando for invocado. Isso ajuda na verificação de tipos durante a compilação e na compreensão do código por parte dos desenvolvedores. No exemplo apresentado, o método **saudacao** não retorna nenhum valor, apenas exibe o resultado, pois para definir o método sem retorno, utilizamos a palavra reservada **void**.

Vamos visualizar o código completo:

```

class Pessoa {

 nome: string;
 idade: number;

 constructor(nome: string, idade: number, genero: string) {
 this.nome = nome;
 this.idade = idade;
 }

 saudacao(): void {
 console.log(`Olá, seu nome é ${this.nome} e sua idade é ${this.idade}
anos.`);
 }

}

```

Com o código completo, vamos transpilar e visualizar o resultado no console.

Os métodos são fundamentais para encapsular comportamentos relacionados em

uma classe e promover a reutilização de código. Eles permitem que a lógica de manipulação de dados seja agrupada de forma coesa com os próprios dados, facilitando a manutenção e compreensão do código.



## Boas práticas de programação!

Analisando o código completo, podemos observar uma boa prática de programação orientada a objetos.

- **Definição dos nomes das classes:** para diferenciar a identificação das classes sempre inicie com letra maiúscula.

- **Nomes de propriedades:** as propriedades sempre devem iniciar com letra minúscula, sem acentuação, espaçamento em branco ou caracteres especiais, mas caso seja necessário separar palavra, opte por identificar o início da segunda palavra com letra maiúscula. Por exemplo, nomePessoa.

- **Objetos:** ao instanciar o objeto de uma classe sempre defina o nome do objeto iniciando com letra minúscula.

Com a definição da classe utilizando suas propriedades (características) e fazendo a primeira instância de um objeto, podemos visualizar no console do navegador seus atributos. Para transpilar o código criado vamos, primeiramente, definir o arquivo **tsconfig.json** para o projeto:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS \EstudoDeClasses> tsc --init

Created a new tsconfig.json with:

 target: es2016
 module: commonjs
 strict: true
 esModuleInterop: true
 skipLibCheck: true
 forceConsistentCasingInFileNames: true

You can learn more at https://aka.ms/tsconfig
```

Ao transpilar o código, é possível identificar que nenhum erro foi encontrado. Com a classe definida e suas propriedades podemos adicionar os métodos.

## 8.4 Modificadores de acesso em TypeScript

Os modificadores de acesso são úteis para **controlar como os membros de uma classe são acessados**. Eles ajudam a manter a segurança dos dados, **esconder detalhes internos de implementação e evitar acesso não autorizado**. Além disso, os modificadores de acesso ajudam a **prevenir erros e bugs** ao restringir o acesso aos membros apenas aos locais onde são necessários.

Em TypeScript, os modificadores de acesso são palavras-chave que controlam a visibilidade e a acessibilidade de membros (propriedades e métodos) de uma classe. **Eles determinam se os membros de uma classe podem ser acessados de dentro da própria classe, de classes derivadas (herdeiras) ou de fora da classe.**

Existem três principais modificadores de acesso em TypeScript:

- **public**: membros marcados como *public* são acessíveis de qualquer lugar, tanto de dentro da classe quanto de fora dela, incluindo classes derivadas. Vamos a um exemplo:

```
class Carro {
 //observe que a propriedade modelo está definida como pública
 public modelo: string;
 constructor(modelo: string) {
 this.modelo = modelo;
 }

 public dirigir(): void {
 console.log(`O ${this.modelo} está dirigindo.`);
 }
}

let meuCarro = new Carro("Ferrari");
console.log(meuCarro.modelo); // Acesso permitido
meuCarro.dirigir(); // Acesso permitido
```

- **private**: membros marcados como *private* são acessíveis somente de dentro da própria classe onde foram definidos. Eles não são acessíveis de fora da classe nem de classes derivadas.

```

class ContaBancaria {
 //Observe que a propriedade saldo tem o acesso privado
 private saldo: number;

 constructor(saldo: number) {
 this.saldo = saldo;
 }

 public verificarSaldo(): void {
 console.log(`Saldo: ${this.saldo}`);
 }
}

let conta = new ContaBancaria(1000);
console.log(conta.saldo); // Erro: Propriedade 'saldo' é privada e só
// acessível dentro da classe
conta.verificarSaldo(); // Acesso permitido

```

- **protected**: membros marcados como *protected* são acessíveis dentro da própria classe e de classes derivadas (herdeiras), mas não são acessíveis de fora da classe ou de instâncias da classe.

```

class Animal {
 protected nome: string;

 constructor(nome: string) {
 this.nome = nome;
 }

 protected fazerBarulho(): void {
 console.log("Barulho genérico");
 }
}

class Cachorro extends Animal {
 latir(): void {
 console.log(`${this.nome} está latindo.`);
 this.fazerBarulho(); // Acesso permitido
 }
}

let cachorro = new Cachorro("Rex");

```

```
cachorro.latir(); // Acesso permitido
console.log(cachorro.nome); // Erro: Propriedade 'nome' é protegida e só
acessível dentro da classe base e classes derivadas
```

Essa capacidade de controle de acesso ajuda a prevenir erros e *bugs* ao **limitar o acesso aos membros apenas aos locais onde são necessários**, além de promover a reutilização de código.

## DESAFIO PRÁTICO

### Criação de classes contendo métodos.

---

#### Descrição

Imagine que você foi contratado para desenvolver um sistema de gestão de alunos para uma escola. Você precisa criar uma classe para representar cada aluno, com informações básicas como nome, idade e turma. Além disso, você precisa de métodos para imprimir os detalhes do aluno e para atualizar a turma do estudante.

---

#### Objetivos

Desenvolver classes:

- definição das propriedades de um objeto;
  - definição dos métodos do objeto;
  - compreender sobre classes para representar objetos do mundo real.
- 

### Orientações

- Desenvolver códigos baseados na orientação a objetos;
- Fazer o uso eficiente da programação orientada a objetos;

- Criar uma classe com os atributos necessários para o desafio.
- Utilizar construtores e métodos que melhorem o código e deixe-o mais eficaz e intuitivo.



## 8.5 Implementação de herança com *extends* e *super*

Em TypeScript, assim como em outras linguagens orientadas a objetos, a herança é um conceito fundamental que permite criar novas classes com base nas já existentes. A classe que é estendida é chamada de classe base ou superclasse, e a classe que herda é conhecida como subclasse ou classe filha. Por exemplo, a classe A deu origem à classe B e C, assim, a classe A é classe base e as B e C são as classes filhas.

Quando uma classe herda de outra, ela recebe todos os membros (métodos e propriedades) da classe base (ou superclasse). Isso significa que a classe derivada pode acessar e usar os membros da classe base, além de adicionar novos membros ou sobrescrever os membros existentes, se necessário.

Para aplicação de herança em TypeScript utilizamos a palavra reservada ***extends***. Vamos visualizar um exemplo utilizando a classe já criada **Pessoa**.

Na classe **Pessoa** temos as propriedades **nome** e **idade**, características que podem ser comuns para qualquer pessoa, sendo professor, aluno, funcionário, entre outros. Mas o aluno terá características específicas, como o número de matrícula. Então, podemos determinar que a classe Pessoa será nossa **superclasse** e as classes com características específicas serão nossas **subclasses**. Cada uma das subclasses poderia herdar características comuns da classe **Pessoa** e adicionar suas próprias características específicas.

Vamos relembrar o nosso exemplo da classe **Pessoa**:

```
class Pessoa {
 nome: string;
 idade: number;

 constructor(nome: string, idade: number) {
 this.nome = nome;
 this.idade = idade;
 }
}
```



```

 saudacao(): void {
 console.log(`Olá, seu nome é ${this.nome} e sua idade é ${this.idade}
anos.`);
 }
}

```

Agora, vamos criar uma subclasse chamada Estudante. Para isso, adicione um novo arquivo ao projeto:

Vamos adicionar o código da declaração da classe **Estudante**:

```

class Estudante extends Pessoa {

```

Importante observar que na declaração já adicionamos a palavra reservada **extends** junto com a identificação da superclasse (neste caso Pessoa) que vai fornecer todas suas propriedades e métodos.

A classe Estudante herda tudo da classe **Pessoa** e adiciona um atributo **matricula**:

```

 private matricula: string;

 constructor(nome: string, idade: number, matricula: string) {
 super(nome, idade);
 this.matricula = matricula;
 }

```

A classe **Estudante** também redefine o método **saudacao ()** para incluir a informação adicional sobre a matrícula do estudante. Para tal, foi utilizado a palavra reservada **super**.

```

 public apresentar(): void {
 super.saudacao();
 console.log(`Minha matrícula é ${this.matricula}.`);
 }
}

```

Vamos visualizar a codificação completa:

```
class Estudante extends Pessoa {
 private matricula: string;

 constructor(nome: string, idade: number, matricula: string) {
 super(nome, idade);
 this.matricula = matricula;
 }

 public apresentar(): void {
 super.saudacao();
 console.log(`Minha matrícula é ${this.matricula}.`);
 }
}
```

## Saiba mais!

Para saber mais sobre aplicação do conceito de herança em TypeScript, assista ao vídeo que apresenta uma visão geral sobre o assunto.



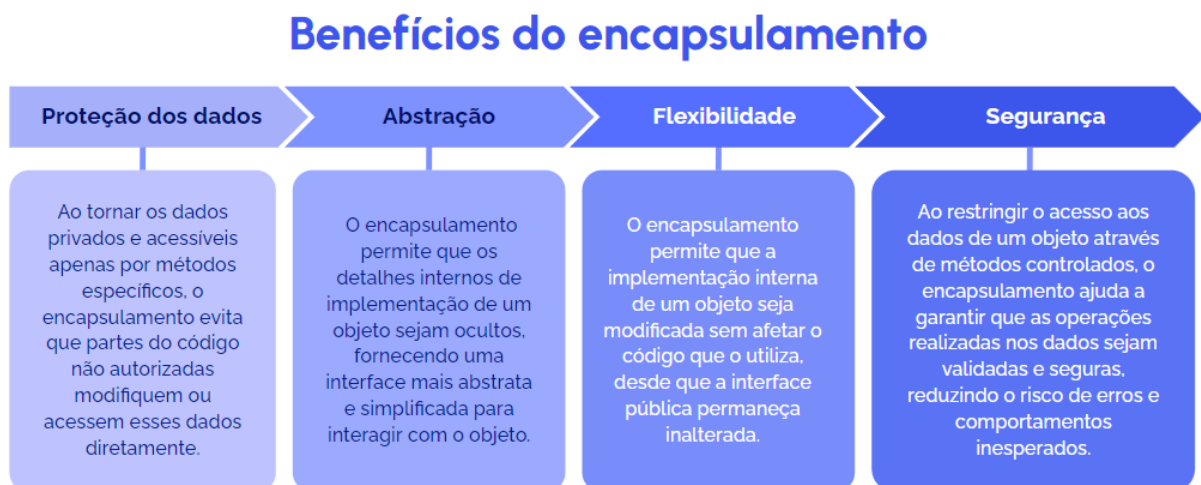
Vídeo Herança de Classes no TypeScript (extends), do canal NBK Mundo Tech. Disponível em: <https://www.youtube.com/watch?v=4znZYwQuaf8>. Acesso em: 20 mar. 2024

## 8.6 Encapsulamento de dados em TypeScript

Em TypeScript, assim como em outras linguagens de programação orientadas a objetos, o encapsulamento de dados é um conceito fundamental que visa controlar o acesso aos dados de um objeto. Isso é feito ocultando os detalhes de implementação e expondo apenas a interface necessária para interagir com o objeto. O encapsulamento permite que os dados de um objeto sejam protegidos e manipulados de forma segura, evitando acessos diretos e não autorizados.

Mas o que significa encapsular dados?

Encapsular dados é um conceito chave na programação orientada a objetos, no qual você agrupa os dados e os métodos que operam nesses dados em uma única unidade, chamada de classe. O encapsulamento de dados ajuda a dividir o código em partes mais gerenciáveis, garantindo a segurança e facilitando a manutenção. Ele também permite criar sistemas mais robustos e adaptáveis, promovendo uma melhor organização e reutilização do código.



Em TypeScript, o encapsulamento de dados é frequentemente alcançado usando os modificadores de acesso: **public**, **protected** e **private**. Com a aplicação de encapsulamento podemos acessar os dados protegidos e os métodos de acesso, também conhecidos como **"getters"** e **"setters"**, para acessar e modificar os dados de um objeto, em vez de permitir o acesso direto às suas variáveis internas. Eles permitem que você controle o acesso aos dados de um objeto, mantendo a coesão e o encapsulamento dos dados.

### Getter

Usado para recuperar o valor de uma propriedade de um objeto. Ele fornece uma forma de acesso somente leitura para as propriedades, permitindo que você obtenha seus valores sem permitir a modificação direta.

Os **getters**, geralmente, possuem o **prefixo *get* seguido pelo nome da propriedade** que desejamos acessar.

X

### Setter

Usado para modificar o valor de uma propriedade de um objeto. Ele fornece uma forma de acesso somente escrita para as propriedades, permitindo que você altere seus valores de acordo com regras ou validações específicas.

Os **setters**, geralmente, possuem o **prefixo *set* seguido pelo nome da propriedade** que desejamos modificar e **seguido por um parâmetro que representa o novo valor da propriedade**.

Vamos visualizar na prática a criação de métodos e encapsulamento de dados? Para isso, vamos modificar a classe **Pessoa** utilizada anteriormente.

Vamos iniciar tornando privado o acesso a propriedades.

```
class Pessoa {
 private nome: string;
 private idade: number;
```

No método, nada foi alterado:

```
 constructor(nome: string, idade: number) {
 this.nome = nome;
 this.idade = idade;
 }
```

Para acessar dados das propriedades, vamos criar os métodos de acesso, método **get**.

```
 public getNome(): string {
 return this.nome;
 }

 public getIdade(): number {
 return this.idade;
 }
```

E para ser possível modificar o valor de uma propriedade, vamos criar os métodos **set**.

```
public setNome(nome: string): void {
 this.nome = nome;
}

public setIdade(idade: number): void {
 this.idade = idade;
}
```

Por fim, vamos visualizar como fica a criação de objetos da classe e o acesso aos dados encapsulados.

```
// Criando um objeto da classe Pessoa
let pessoa = new Pessoa("João", 30);

// Acesso aos dados encapsulados
console.log(pessoa.getNome()); // Saída: João
console.log(pessoa.getIdade()); // Saída: 30
```

Vamos visualizar o código completo:

```
class Pessoa {
 private nome: string;
 private idade: number;

 constructor(nome: string, idade: number) {
 this.nome = nome;
 this.idade = idade;
 }

 public getNome(): string {
 return this.nome;
 }

 public getIdade(): number {
 return this.idade;
 }
}
```

```

 public setNome(nome: string): void {
 this.nome = nome;
 }

 public setIdade(idade: number): void {
 this.idade = idade;
 }

 saudacao(): void {
 console.log(`Olá, seu nome é ${this.nome} e sua idade é ${this.idade}
anos.`);
 }
}

// Criando um objeto da classe Pessoa
let pessoa = new Pessoa("João", 30);

// Acesso aos dados encapsulados
console.log(pessoa.getNome()); // Saída: João
console.log(pessoa.getIdade()); // Saída: 30

```

### Vamos lembrar?

Agora que já conhecemos os conceitos de herança e em encapsulamento, vamos desenvolver um exemplo prático envolvendo os dois conceitos. Vamos criar uma hierarquia de classes para representar diferentes tipos de animais. As classes terão propriedades encapsuladas e métodos para acessá-las. Além disso, usaremos herança para representar relações entre os diferentes tipos de animais.

Vamos iniciar definindo um novo arquivo TypeScript denominado ***Animal***.

```

// Definição da classe Animal (classe base)
class Animal {
 private _nome: string;
 private _idade: number;

```

```

constructor(nome: string, idade: number) {
 this._nome = nome;
 this._idade = idade;
}

```

Observe que as propriedades foram definidas como **private**, portanto temos que adicionar os métodos **getter** e **setter** para acesso.

```

// Getter para o nome
public get nome(): string {
 return this._nome;
}

// Setter para o nome
public set nome(novoNome: string) {
 this._nome = novoNome;
}

// Getter para a idade
public get idade(): number {
 return this._idade;
}

// Setter para a idade
public set idade(novaIdade: number) {
 this._idade = novaIdade;
}

```

Declarado as propriedades da classe, vamos à definição do primeiro método.

```

// Método para fazer o animal emitir um som
public emitirSom(): void {
 console.log("O animal emitiu um som.");
}
}

```

A classe **Animal** será utilizada como a superclasse do projeto e, a partir dela, serão criadas as subclasses que herdarão os dados da superclasse e terão suas propriedades e métodos próprios. A primeira subclasse a ser criada será a **Cachorro**, que tem como propriedade **raca**.

```
// Definição da classe Cachorro (subclasse de Animal)
class Cachorro extends Animal {
 private _raca: string;

 constructor(nome: string, idade: number, raca: string) {
 super(nome, idade);
 this._raca = raca;
 }
}
```

Vamos adicionar os métodos de acesso.

```
// Getter para a raça
public get raca(): string {
 return this._raca;
}

// Setter para a raça
public set raca(novaRaca: string) {
 this._raca = novaRaca;
}
```

Agora, vamos definir um método específico da classe **Cachorro**.

```
// Método para fazer o cachorro latir
public latir(): void {
 console.log("Au Au!");
}
}
```

Por fim, vamos visualizar exemplos de uso da classe:

```
// Exemplo de uso das classes
let animal = new Animal("Animal", 5);
console.log(animal.nome); // Saída: Animal
console.log(animal.idade); // Saída: 5
animal.emitirSom(); // Saída: O animal emitiu um som.

let cachorro = new Cachorro("Rex", 3, "Labrador");
console.log(cachorro.nome); // Saída: Rex
```



```
console.log(cachorro.idade); // Saída: 3
console.log(cachorro.raca); // Saída: Labrador
cachorro.emitirSom(); // Saída: O animal emitiu um som.
cachorro.latir(); // Saída: Au Au!
```

Neste exemplo, temos duas classes: **Animal** e **Cachorro**. A classe **Cachorro** herda da classe **Animal**. Ambas as classes têm propriedades encapsuladas (`_nome` e `_idade` para **Animal** e `_raca` para **Cachorro**) e métodos *getters* e *setters* para acessá-las. Além disso, cada classe tem um método específico (`emitirSom` para **Animal** e `latir` para **Cachorro**). Isso demonstra o uso de encapsulamento e herança em TypeScript.

## DESAFIO PRÁTICO

### Criação de classes envolvendo encapsulamento e herança.

---

#### Descrição

Você está desenvolvendo um sistema de gerenciamento de funcionários para uma empresa com diferentes tipos de funcionários, incluindo funcionários em tempo integral e em meio período. Todos com nome, número de identificação único e um salário-base.

Os funcionários em tempo integral têm um salário mensal fixo, enquanto os funcionários em meio período recebem por hora trabalhada. Além disso, os funcionários em tempo integral podem receber bônus anuais com base no desempenho, enquanto os funcionários em meio período não recebem bônus. Crie um código que cumpra com todas essas expectativas da empresa, utilizando o conhecimento desenvolvido neste capítulo.

---

#### Objetivos

Desenvolver classes:

- definição das propriedades de um objeto;

- definição dos métodos do objeto.;
- aplicação dos conceitos de herança e encapsulamento.

---

## Orientações

- Desenvolva códigos baseados na orientação a objetos;
- Crie uma classe base e outras subclasses com as características necessárias para atender ao pedido da empresa;
- Implemente métodos, instâncias e faça o encapsulamento dos dados para tornar o código mais organizado e compreensível.



Aqui, vimos alguns tópicos essenciais para compreender a estrutura e o funcionamento das classes, falando sobre sintaxe básica para a declaração de classes em TypeScript, destacando a importância de especificar o tipo de dados dos membros da classe. Em seguida, exploramos as propriedades de uma classe, ressaltando a necessidade de definir os tipos de dados das propriedades para garantir a consistência e segurança do código. Além disso, vimos os métodos de uma classe, sendo eles as funções que podem ser chamadas nos objetos da classe para realizar operações específicas, enfatizando a importância de declarar os parâmetros e o tipo de retorno dos métodos para evitar erros durante a execução do programa. Outro ponto fundamental foram os modificadores de acesso em TypeScript, que são palavras-chave como *public*, *private* e *protected*, utilizadas para controlar o acesso aos membros da classe. Isso permite garantir a encapsulação dos dados e evitar acesso não autorizado. Vimos como ocorre a implementação de herança com *extends* e *super*, mostrando como uma classe pode herdar propriedades e métodos de outra classe. Por fim, foi discutido o conceito de encapsulamento de dados, que consiste em ocultar os detalhes de implementação dos membros da classe e expor apenas as interfaces necessárias para o uso externo, o que contribui para a modularidade e reusabilidade do código.



## ATIVIDADE DE FIXAÇÃO

1. Explique o conceito de classes em TypeScript e como elas são usadas para modelar objetos. Dê um exemplo de uma classe simples em TypeScript e explique os seus componentes.
2. Descreva um cenário de aplicação do mundo real em que você usaria classes e métodos em TypeScript para resolver um problema específico. Discuta como você projetaria as classes e métodos para atender aos requisitos desse cenário.
3. Explique o que é encapsulamento em programação orientada a objetos. Por que o encapsulamento é importante ao projetar classes em TypeScript?
4. Discuta como o encapsulamento de dados contribui para a segurança e integridade do código. Como o encapsulamento ajuda a prevenir acesso não autorizado e manipulação indevida dos dados de um objeto?
5. Explique o que é herança em programação orientada a objetos.
6. Como a herança é implementada em TypeScript e qual é a sua importância na criação de hierarquias de classes?
7. Quais são os modificadores de acesso disponíveis para membros de uma classe em TypeScript e como eles afetam a herança?
8. Qual é a importância do encapsulamento em programação orientada a objetos?
9. Como você implementaria herança em TypeScript? Dê um exemplo de uma hierarquia de classes que demonstre o uso de herança e explique como isso facilita a reutilização de código e a organização de classes.
10. Descreva um cenário de aplicação do mundo real em que você usaria classes e métodos em TypeScript para resolver um problema específico.

# CAPÍTULO 09

## MÓDULOS E IMPORTAÇÕES EM TYPESCRIPT

### O que esperar deste capítulo:

- Compreender a definição de módulos e exportação de código;
- Conseguir realizar a importação de módulos em outros arquivos TypeScript;
- Entender como aplicar a resolução de caminhos de módulos com configuração de `tsconfig.json`;
- Saber como utilizar módulos em TypeScript.

### 9.1 O que são módulos?

Módulos são arquivos que guardam o código que você escreve sobre um assunto ou uma função específica. Por exemplo, você pode ter um módulo chamado `math.ts`, que possui funções matemáticas, e outro módulo chamado `user.ts`, que tem uma classe que representa um usuário.

Os módulos te ajudam a **separar** o código em pedaços menores e mais fáceis de lidar, evitando que o código fique muito grande e confuso. Além disso, os módulos te permitem **reaproveitar** o código em diferentes partes do projeto ou, até mesmo, em outros projetos, sem precisar copiar e colar o código.

Você pode **usar módulos para separar o seu código em arquivos que contêm código relacionado a um determinado assunto ou funcionalidade**. Por exemplo, você pode ter um módulo que contém funções para manipular *strings*, e outro módulo que contém uma classe para representar um produto.

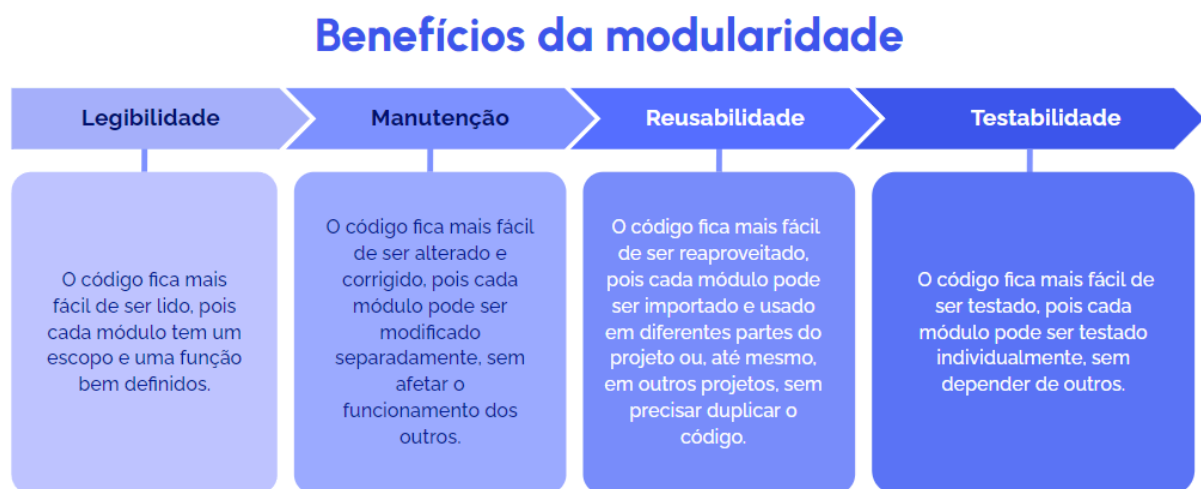
Para trabalhar com módulos em TypeScript, você precisa saber como criar, exportar e importá-los, além de saber como usar módulos de terceiros, como bibliotecas ou *frameworks*. Além disso, você precisa conhecer como organizar os seus módulos em uma estrutura de pastas que mostre a relação entre eles e como configurar o seu projeto para resolver os caminhos de módulos. Que tal aprendermos um pouco mais sobre?

## 9.2 Como organizar o código em módulos?

Uma dica para organizar o código em módulos é seguir o princípio da **responsabilidade única**, ou seja, cada módulo deve ter só uma responsabilidade ou objetivo. Por exemplo, se você tem um módulo que tem funções para validar dados, não misture essas funções com outras que fazem cálculos.

Outra dica é usar uma **estrutura de pastas** que mostre a organização dos módulos. Por exemplo, você pode ter uma pasta chamada *services*, que possui módulos que conversam com APIs externas, ou seja, questões voltadas para coisas fora do código principal, e outra pasta chamada *models*, que possui módulos que definem as classes ou interfaces dos dados do projeto, ou seja, voltada para dados internos do projeto.

A estrutura de pastas pode mudar de acordo com o tipo e o tamanho do projeto, mas o importante é que ela seja **consistente** e **intuitiva**, facilitando a localização e a manutenção dos módulos.



## 9.3 Como importar e exportar um módulo no TypeScript

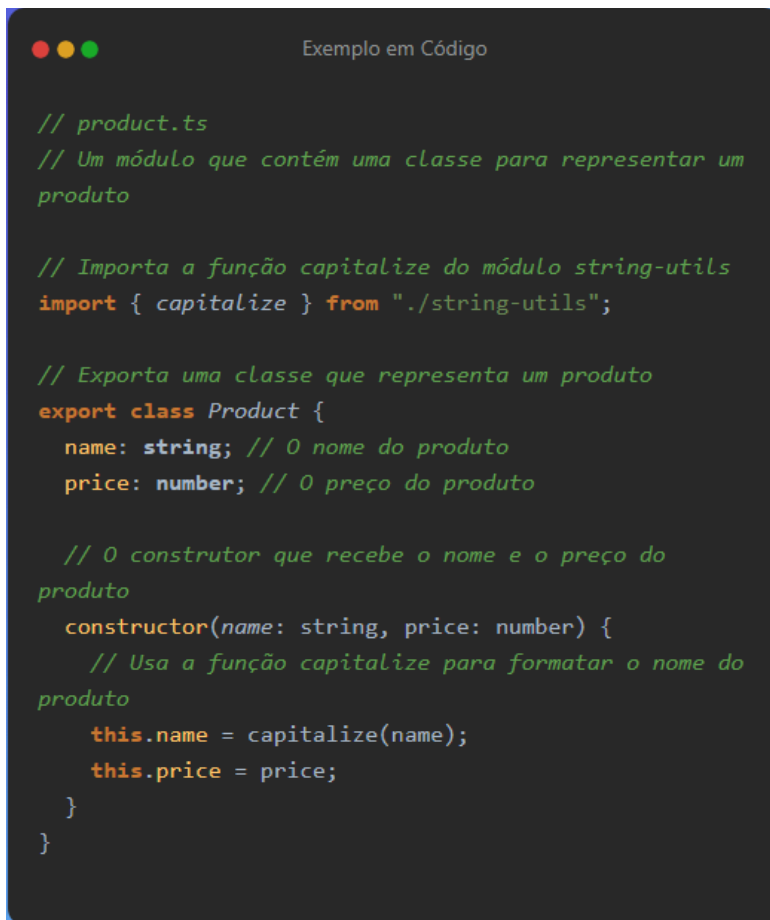
Para criar um módulo no TypeScript, que possa ser utilizado em outros arquivos, é só usar o **"export"** para dizer quais partes do módulo quer compartilhar com outros módulos. Se quiser dividir uma função chamada "capitalize", você escreve assim:

Exemplo em Código

```
// string-utils.ts
// Um módulo que contém funções para manipular strings

// Exporta uma função que converte a primeira letra de
// uma string em maiúscula
export function capitalize(str: string) {
 return str.charAt(0).toUpperCase() + str.slice(1);
}
```

Caso você queira usar um elemento de um módulo em outro módulo, você precisa usar a palavra-chave **import** para indicar de qual módulo e qual elemento você quer importar. Se quiser importar a função "capitalize" do módulo **string-utils.ts** em outro módulo chamado **product.ts**, escreva o código a seguir:

A imagem mostra um editor de código com o título "Exemplo em Código". O código é escrito em TypeScript e demonstra a importação e exportação de uma função e uma classe. O código está colorido: comentários em verde, palavras-chave em laranja e nomes de variáveis ou classes em branco. O código é o seguinte:

```
// product.ts
// Um módulo que contém uma classe para representar um
// produto

// Importa a função capitalize do módulo string-utils
import { capitalize } from "../string-utils";

// Exporta uma classe que representa um produto
export class Product {
 name: string; // O nome do produto
 price: number; // O preço do produto

 // O construtor que recebe o nome e o preço do
 // produto
 constructor(name: string, price: number) {
 // Usa a função capitalize para formatar o nome do
 // produto
 this.name = capitalize(name);
 this.price = price;
 }
}
```

A exportação de código em TypeScript é um recurso que permite compartilhar funções, classes, interfaces, variáveis e outros elementos entre diferentes módulos ou arquivos. Já a importação será uma ação feita no mesmo código, mas em outras partes deles. Isso facilita a organização e a reutilização do código.

A sintaxe para exportar um elemento em TypeScript é simples. Você pode usar a palavra-chave **export** - antes da declaração do elemento ou usar a palavra-chave **export** - seguida de chaves "{}" para exportar vários elementos de uma vez.

Veja o exemplo seguir. Você pode exportar uma função chamada *sum* e uma variável chamada *PI* de um módulo chamado *math.ts*, das seguinte formas:

```
// math.ts
// Um módulo que contém elementos relacionados à matemática

// Exporta uma função que soma dois números
export function sum(a: number, b: number) {
 return a + b;
}

// Exporta uma variável que guarda o valor de PI
export const PI = 3.14;
```

Podemos também tratar de outra forma

```
// math.ts
// Um módulo que contém elementos relacionados à matemática

// Declara uma função que soma dois números
function sum(a: number, b: number) {
 return a + b;
}

// Declara uma variável que guarda o valor de PI
const PI = 3.14;

// Exporta a função e a variável
export { sum, PI };
```

Você também pode usar a palavra-chave **default** para indicar que um elemento é o exportado por padrão do módulo, ou seja, caso não seja especificado no *export* qual elemento será importado, esse que foi definido como padrão é que será utilizado. **Você só pode ter um elemento padrão exportado por módulo.** Veja o exemplo, você pode exportar a função *sum* como o elemento padrão do módulo *math.ts*, da seguinte forma:

```
// math.ts
// Um módulo que contém elementos relacionados à matemática

// Exporta uma função que soma dois números como o elemento padrão
export default function sum(a: number, b: number) {
 return a + b;
}
```

```
// Exporta uma variável que guarda o valor de PI
export const PI = 3.14;
```

podendo ser tratada também da seguinte forma:

```
// math.ts
// Um módulo que contém elementos relacionados à matemática

// Declara uma função que soma dois números
function sum(a: number, b: number) {
 return a + b;
}

// Declara uma variável que guarda o valor de PI
const PI = 3.14;

// Exporta a função como o elemento padrão e a variável
export default sum;
export { PI };
```



### Exportação padrão

não requer um nome para o elemento exportado e **pode ser importada usando qualquer nome**.

X

### Exportação nomeada

requer um nome para o elemento exportado e **deve ser importada usando o mesmo nome** ou um nome alternativo usando a palavra-chave **as**.

Para ilustrar a diferença, vamos ver como importar os elementos exportados do módulo `math.ts` em outro módulo chamado `app.ts`. Se usarmos a exportação nomeada, podemos importar os elementos da seguinte forma:

```
// app.ts
// Um módulo que usa os elementos do módulo math

// Importa a função sum e a variável PI do módulo math
import { sum, PI } from "./math";

// Chama a função sum com os argumentos 1 e 2 e imprime o resultado
console.log(sum(1, 2)); // 3

// Imprime o valor de PI
console.log(PI); // 3.14
```

Se quisermos usar um nome diferente para os elementos importados, podemos usar a palavra-chave **para** e assim renomeá-los. Por exemplo, podemos importar a função `sum` como `add` (`sum as add`) e a variável `PI` como `pi` (`PI as pi`), da seguinte forma:

```
// app.ts
// Um módulo que usa os elementos do módulo math

// Importa a função sum como add e a variável PI como pi do módulo math
import { sum as add, PI as pi } from "./math";
```

```
// Chama a função add com os argumentos 1 e 2 e imprime o resultado
console.log(add(1, 2)); // 3

// Imprime o valor de pi
console.log(pi); // 3.14
```

Já se queremos usar a exportação padrão, podemos importar o elemento padrão usando qualquer nome, sem usar chaves. Por exemplo, podemos importar a função *sum* como *add* e a variável *PI* como *pi* (**PI as pi**), da seguinte forma:

```
// app.ts
// Um módulo que usa os elementos do módulo math

// Importa a função padrão como add e a variável PI como pi do módulo math
import add, { PI as pi } from "./math";

// Chama a função add com os argumentos 1 e 2 e imprime o resultado
console.log(add(1, 2)); // 3

// Imprime o valor de pi
console.log(pi); // 3.14
```

Você também pode **importar todos os elementos de um módulo** usando o operador `*` e um nome para o módulo. Por exemplo, podemos importar todos os elementos do módulo `math` como `math`, usando a seguinte forma:

```
/ app.ts
// Um módulo que usa os elementos do módulo math

// Importa todos os elementos do módulo math como math
import * as math from "./math";

// Chama a função sum com os argumentos 1 e 2 e imprime o resultado
```

```
console.log(math.sum(1, 2)); // 3

// Imprime o valor de PI
console.log(math.PI); // 3.14
```

## 9.4 Resolução de caminhos de módulos com configuração de tsconfig.json

A resolução de caminhos de módulos é **o processo de encontrar o arquivo que contém o módulo que você quer importar em seu código**, funcionando como a opção buscar que você utiliza para achar uma palavra em um texto. Em TypeScript, você pode usar o arquivo **tsconfig.json** para configurar a forma como o compilador resolve os caminhos de módulos, usando a propriedade **paths**.

A propriedade **paths** permite que sejam **definidas aliases para os caminhos de módulos**, ou seja, **nomes alternativos que você pode usar para se referir aos módulos** sem precisar escrever o caminho completo. Caso você possua um **módulo chamado user.ts na pasta src/models**, pode **definir um alias chamado @models/user** para esse módulo, usando a seguinte configuração:

```
// tsconfig.json
{
 "compilerOptions": {
 // Outras opções de compilação
 "baseUrl": "./", // Define a base para os caminhos relativos
 "paths": {
 "@models/*": ["src/models/*"]
 }
 }
}

// Define um alias para os módulos na pasta src/models
```

Com essa configuração, o módulo `user.ts` pode ser importado usando o alias `@models/user`, ao invés de usar o caminho relativo `../src/models/user` dessa forma:

```
// app.ts
// Um módulo que usa o módulo user

// Importa o módulo user usando o alias @models/user
import { User } from "@models/user";

// Cria um objeto da classe User
let user = new User("Alice", 25);

// Imprime o nome e a idade do usuário
console.log(user.name); // Alice
console.log(user.age); // 25
```

Essa forma de importar módulos usando aliases **simplifica o código**, pois você não precisa se preocupar com o caminho relativo do módulo e pode usar um nome mais curto e intuitivo. Além disso, essa forma de importar módulos **facilita a manutenção do código**, visto que, se você mudar a localização do módulo, só precisará atualizar o alias no arquivo `tsconfig.json`, e não em todos os lugares que você importou o módulo.

Pode-se definir quantos aliases você quiser e usar padrões com asteriscos (\*) para abranger vários módulos. Como exemplo, **você pode definir um alias chamado `@services/*` para os módulos na pasta `src/services`**, usando a seguinte configuração:

```
// tsconfig.json
{
 "compilerOptions": {
 // Outras opções de compilação
 "baseUrl": "./", // Define a base para os caminhos relativos
 "paths": {
 "@models/*": ["src/models/*"],
 // Define um alias para os módulos na pasta src/models
 "@services/*": ["src/services/*"]
 }
 }
}
```

```
// Define um alias para os módulos na pasta src/services
}
}
}
```

Com essa configuração, você pode importar qualquer módulo na pasta `src/services` usando o alias `@services/`, seguido pelo nome do módulo. Caso tenha um módulo chamado `auth.ts` na pasta `src/services`, pode importá-lo usando o alias `@services/auth`, em vez de usar o caminho relativo `./src/services/auth`. Pode ser feito dessa forma:

```
// app.ts
// Um módulo que usa o módulo auth

// Importa o módulo auth usando o alias @services/auth
import { login, logout } from "@services/auth";

// Chama as funções login e logout
login("alice", "1234"); // Faz o login do usuário alice com a senha 1234
logout(); // Faz o logout do usuário
```

Essa é a forma de configurar e usar a propriedade `paths` no arquivo `tsconfig.json` para resolver os caminhos de módulos em TypeScript. **Sendo assim, pode-se usar essa forma para definir aliases para os módulos que você quer importar e simplificar as importações com nomes mais curtos e intuitivos.**

## 9.5 Casos de uso prático

Uma forma prática de trabalhar com módulos em TypeScript é criar uma aplicação *web* que usa o *framework* Angular. **O Angular é um *framework* que facilita o desenvolvimento de aplicações *web* dinâmicas e interativas, usando o padrão MVC (*Model-View-Controller*).**

O Angular usa módulos para organizar o código da aplicação em componentes, serviços, diretivas, *pipes* e outros elementos. Cada elemento é definido em um arquivo TypeScript, que é exportado e importado por outros arquivos. O Angular também usa

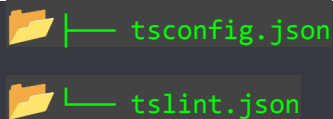
módulos de terceiros, como o RxJS, que é uma biblioteca que permite trabalhar com programação reativa e assíncrona.

Para criar uma aplicação *web* com o Angular, você precisa usar o **Angular CLI**, uma ferramenta que gera e configura o seu projeto. **O Angular CLI cria uma estrutura de pastas que contém os módulos da sua aplicação e um arquivo `tsconfig.json`** que configura o compilador TypeScript e a resolução de caminhos de módulos.

Por exemplo, se você usar o Angular CLI para criar uma aplicação chamada `my-app`, você vai ter uma estrutura de pastas como esta:

```
my-app
├── e2e
├── node_modules
├── src
│ ├── app
│ │ ├── app.component.css
│ │ ├── app.component.html
│ │ ├── app.component.spec.ts
│ │ ├── app.component.ts
│ │ └── app.module.ts
│ ├── assets
│ ├── environments
│ ├── index.html
│ ├── main.ts
│ ├── polyfills.ts
│ ├── styles.css
│ └── test.ts
├── angular.json
└── package.json
```

O diagrama mostra a estrutura de pastas de um projeto Angular. No topo, há uma pasta principal chamada `my-app`. Dentro dela, há subpastas para `e2e`, `node_modules`, `src`, `assets`, `environments`, `index.html`, `main.ts`, `polyfills.ts`, `styles.css`, `test.ts`, `angular.json` e `package.json`. A pasta `src` contém uma subpasta `app` que, por sua vez, contém arquivos `app.component.css`, `app.component.html`, `app.component.spec.ts`, `app.component.ts` e `app.module.ts`.



Nessa estrutura, o arquivo **app.module.ts** é o módulo principal da aplicação que **importa e declara os outros módulos e elementos que a aplicação usa**. O arquivo `app.module.ts` pode ter o seguinte conteúdo:

```
// app.module.ts
// O módulo principal da aplicação

// Importa os módulos do Angular
import { BrowserModule } from "@angular/platform-browser";
import { NgModule } from "@angular/core";

// Importa o módulo do componente app
import { AppComponent } from "./app.component";

// Decora a classe AppModule com o decorador NgModule
@NgModule({
 declarations: [
 AppComponent // Declara o componente app
],
 imports: [
 BrowserModule // Importa o módulo do navegador
],
 providers: [], // Define os provedores de serviços
 bootstrap: [AppComponent]
 // Define o componente que inicia a aplicação
})
// Exporta a classe AppModule
export class AppModule {}
```

O arquivo `app.component.ts` é o **módulo do componente app** que define a **lógica e os dados do componente**. O arquivo `app.component.ts` pode ter o seguinte conteúdo:

```
// app.component.ts
// O módulo do componente app

// Importa o módulo do Angular
import { Component } from "@angular/core";

// Decora a classe AppComponent com o decorador Component
@Component({
 selector: "app-root", // Define
```



Conhecemos um pouco sobre os módulos em TypeScript, que são como pacotes de código organizados cada um focado em um tema ou funcionalidade específica. Agora, você sabe como **criar, exportar e importar módulos**, utilizando as palavras-chave **export** e **import**, entende a distinção entre **exportação padrão e nomeada** e sabe como utilizar a palavra-chave **default** para **indicar o elemento principal do módulo**. Além disso, entende como **lidar com conflitos de nomes entre elementos importados**, aplicando a palavra-chave **as** para renomeá-los ou optando por nomes qualificados, que incluem o nome do módulo. Você também aprendeu como **personalizar a resolução de caminhos de módulos no arquivo tsconfig.json**, utilizando a propriedade **paths** para **criar aliases**, simplificando as importações e tornando nosso código mais elegante. E, por fim, foram apresentados casos de uso prático, desde a criação de uma **aplicação web robusta com Angular** até o **desenvolvimento de uma biblioteca de funções matemáticas reutilizáveis**, usando algumas boas práticas para a organização de módulos, como **seguir o princípio da responsabilidade única, escolher nomes claros e consistentes e documentar os módulos com comentários** proporcionando clareza e facilitando futuras manutenções.





## ATIVIDADE DE FIXAÇÃO

1. Crie um módulo chamado `math.ts` que exporte uma função chamada `sum` que recebe dois números como parâmetros e retorna a soma deles.
2. Crie um módulo chamado `app.ts` que importe a função `sum` do módulo `math.ts` e use-a para imprimir a soma de 1 e 2 no console.
3. Crie um módulo chamado `users.ts` que exporte uma interface chamada `user` que defina as propriedades `name` e `age` de um usuário, ambas do tipo `string`.
4. Crie um módulo chamado `app.ts` que importe a interface `user` do módulo `users.ts` e crie um objeto que implemente essa interface com o nome Alice e a idade 25. Em seguida, imprima as propriedades desse objeto no console.
5. Crie um módulo chamado `math.ts` que exporte duas funções: uma chamada `sum`, que recebe dois números como parâmetros e retorna a soma deles, e outra chamada `product`, que recebe dois números como parâmetros e retorna o produto deles. Em seguida, exporte a função `sum` como o elemento padrão do módulo.
6. Crie um módulo chamado `app.ts` que importe a função padrão do módulo `math.ts` como `add`, e a função `product` do mesmo módulo como `mul`. Em seguida, use essas funções para imprimir a soma e o produto de 2 e 3 no console.
7. Crie um módulo chamado `math.ts` que exporte uma classe chamada `calculator`, que tem um método estático chamado `sum` e que recebe dois números como parâmetros e retorna a soma deles.
8. Crie um módulo chamado `app.ts` que importe a classe `calculator` do módulo `math.ts` e use o método `sum` para imprimir a soma de 4 e 5 no console.
9. Crie um módulo chamado `math.ts` que exporte uma função chamada `sum` que recebe dois números como parâmetros e retorna a soma deles. Em seguida, crie um módulo chamado `stats.ts` que exporte uma função chamada `sum` que recebe

um *array* de números como parâmetro e retorna a soma de todos os elementos do *array*.

10. Crie um módulo chamado `app.ts` que importe as funções `sum` dos módulos `math.ts` e `stats.ts`, usando nomes diferentes para evitar conflitos. Em seguida, use essas funções para imprimir a soma de 6 e 7, e a soma do *array* `[1, 2, 3, 4, 5]` no console.

# CAPÍTULO 10

## GENERICIS E EXTENDS NO TYPESCRIPT

O que esperar deste capítulo:

- Compreender o que é *generics*, sua sintaxe e como utilizá-lo no TypeScript;
- Desenvolver funções e classes genéricas;
- Entender como criar restrições de tipos com *extends*.

### 10.1 Generics em TypeScript

**Generics** é um recurso que permite criar componentes que podem funcionar com vários tipos de dados, em vez de um só. Por exemplo, você pode criar uma função que recebe um array de qualquer tipo e retorna o primeiro elemento desse array usando *generics*.

Utilizar *generics* em TypeScript é uma forma de **aumentar a flexibilidade e a reutilização do código**, permitindo que você crie funções, classes, interfaces ou tipos que podem trabalhar com diferentes tipos de dados sem perder a segurança e a consistência do tipo.

Os *generics* são indicados por um nome entre **colchetes angulares** (<>), que pode ser qualquer identificador válido em TypeScript, mas, geralmente, é uma letra maiúscula, como T, U, V etc. Por exemplo, você pode definir uma função genérica chamada **identity** que **recebe um parâmetro do tipo T e retorna um valor do mesmo tipo T**, usando a seguinte forma:

```
// Define uma função genérica chamada identity que recebe um parâmetro do tipo T e retorna um valor do mesmo tipo T
function identity<T>(value: T): T {
 // Retorna o valor recebido como parâmetro
 return value;
}
```

Com essa definição, você pode **chamar a função identity com qualquer tipo de valor e o tipo T será inferido pelo compilador**, de acordo com o argumento passado. Por exemplo, você pode fazer isso:

```
// Chama a função identity com um valor do tipo string
// O tipo T é inferido como string
let str = identity("Hello"); // str é do tipo string

// Chama a função identity com um valor do tipo number
// O tipo T é inferido como number
let num = identity(42); // num é do tipo number

// Chama a função identity com um valor do tipo boolean
// O tipo T é inferido como boolean
let bool = identity(true); // bool é do tipo boolean
```

Você também pode especificar explicitamente o tipo T ao chamar a função identity, usando a sintaxe de tipo de argumento (<T>), antes dos parênteses. Por exemplo, você pode fazer isso:

```
// Chama a função identity com um valor do tipo string, especificando o tipo T
como string
let str = identity<string>("Hello"); // str é do tipo string

// Chama a função identity com um valor do tipo number, especificando o tipo T
como number
let num = identity<number>(42); // num é do tipo number

// Chama a função identity com um valor do tipo boolean, especificando o tipo
T como boolean
let bool = identity<boolean>(true); // bool é do tipo boolean
```

Essa forma de especificar o tipo T é útil quando o compilador não consegue inferir o tipo T automaticamente ou quando você quer restringir o tipo T a um subtipo específico.

## 10.2 Aplicações do Generics

Você já sabe que a sintaxe para definir um tipo genérico em TypeScript é bastante simples. Você utiliza **par de colchetes angulares (<>)** após o nome do tipo ou da função, seguido por um identificador que será usado para se referir ao tipo genérico. Veja no exemplo, podemos definir um tipo genérico chamado `Array<T>`, que representa um *array* de elementos do tipo `T`:

```
type Array<T> = T[];
```

Assim, você pode usar o tipo genérico `Array<T>` para declarar variáveis ou parâmetros que podem receber *arrays* de qualquer tipo. Por exemplo, você pode declarar uma variável chamada `numbers` que pode receber um *array* de números, usando o tipo genérico `Array<number>`:

```
let numbers: Array<number> = [1, 2, 3];
```

Você também pode usar o tipo genérico `Array<T>` para **definir uma função que recebe um *array* de qualquer tipo e retorna o primeiro elemento desse *array*** usando o tipo genérico `T`:

```
function first<T>(array: Array<T>): T {
 return array[0];
}
```

Você também pode chamar a função `first` com *arrays* de **diferentes tipos** e o **tipo genérico `T` será inferido pelo compilador**, de acordo com o tipo do argumento que você passar. Por exemplo, podemos chamar a **função `first` com um *array* de números e o tipo genérico `T` será inferido como `number`**:

```
let n = first(numbers); // n é do tipo number
```

Você também pode chamar a **função first com um array de strings**, e o tipo genérico **T** será inferido como *string*:

```
let s = first(["a", "b", "c"]); // s é do tipo string
```

Mas também podemos **especificar o tipo genérico T** explicitamente usando a notação **<T>** após o nome da função. Por exemplo, a **função first com um array de números e especificar o tipo genérico T como number**:

```
let n = first<number>(numbers); // n é do tipo number
```

Você também pode **usar mais de um tipo genérico, separando-os por vírgula**. Por exemplo, podemos definir uma função que recebe dois parâmetros de tipos genéricos diferentes, e retorna um objeto que contém esses dois parâmetros, usando os tipos genéricos **T** e **U**:

```
function pair<T, U>(a: T, b: U): { first: T; second: U } {
 return { first: a, second: b };
}
```

É possível também chamar a função **pair** com parâmetros de diferentes tipos e os tipos genéricos **T** e **U** serão inferidos pelo compilador, de acordo com os tipos dos argumentos que você passar. Podemos **chamar a função pair com um número e uma string e os tipos genéricos T e U serão inferidos como number e string**:

```
let p = pair(1, "a"); // p é do tipo { first: number; second: string }
```

**Também podemos especificar os tipos genéricos T e U explicitamente** usando a notação **<T, U>** após o nome da função. Por exemplo, podemos chamar a função **pair** com um número e uma *string*, e especificar os tipos genéricos **T** e **U** como **number** e **string**:

```
let p = pair<number, string>(1, "a");

// p é do tipo
{first: number; second: string }
```

Você também pode usar tipos genéricos para definir classes que podem trabalhar com diferentes tipos de dados. Por exemplo, definir uma classe que representa uma pilha, que pode armazenar elementos de qualquer tipo, usando o tipo genérico T:

```
class Stack<T> {

 private items: Array<T>; // Um array de elementos do tipo T

 private size: number; // O tamanho da pilha

 // O construtor que recebe o tamanho máximo da pilha
 constructor(maxSize: number) {

 this.items = []; // Inicializa o array de elementos como vazio

 this.size = maxSize; // Atribui o tamanho máximo da pilha
 }

 // Um método que empilha um elemento na pilha, se ela não estiver cheia
 push(item: T): void {

 if (this.items.length < this.size) {

 this.items.push(item); // Adiciona o elemento no final do array
 } else {

 throw new Error("Stack is full"); // Lança um erro se a pilha estiver
 cheia
 }
 }
}
```

```

 }

 }

 // Um método que desempilha um elemento da pilha, se ela não estiver vazia

 pop(): T {

 if (this.items.length > 0) {

 return this.items.pop(); // Remove e retorna o último elemento do
array

 } else {

 throw new Error("Stack is empty"); // Lança um erro se a pilha estiver
vazia

 }

 }

}

// Um método que retorna o elemento do topo da pilha, sem removê-lo

peek(): T {

 if (this.items.length > 0) {

 return this.items[this.items.length - 1]; // Retorna o último elemento
do array

 } else {

 throw new Error("Stack is empty"); // Lança um erro se a pilha estiver
vazia

 }

}

```



```

 }

 // Um método que retorna se a pilha está vazia ou não
 isEmpty(): boolean {
 return this.items.length === 0; // Retorna verdadeiro se o array estiver
vazio
 }

 // Um método que retorna o número de elementos na pilha
 length(): number {
 return this.items.length; // Retorna o tamanho do array
 }
}

```

Você pode usar a classe genérica `Stack<T>` para criar objetos que podem armazenar elementos de qualquer tipo. Por exemplo, criar uma pilha de números usando o tipo genérico `number`:

```

let stack = new Stack<number>(10); // Cria uma pilha de números com tamanho
máximo 10

stack.push(1); // Empilha o número 1
stack.push(2); // Empilha o número 2
stack.push(3); // Empilha o número 3

console.log(stack.pop()); // Desempilha e imprime o número 3
console.log(stack.peek()); // Imprime o número 2, sem desempilhar
console.log(stack.length()); // Imprime o número 2, que é o tamanho da pilha

```

Você também pode criar uma pilha de *strings*, usando o tipo genérico `string`:

```
let stack = new Stack<string>(10); // Cria uma pilha de strings com tamanho máximo 10

stack.push("a"); // Empilha a string "a"

stack.push("b"); // Empilha a string "b"

stack.push("c"); // Empilha a string "c"

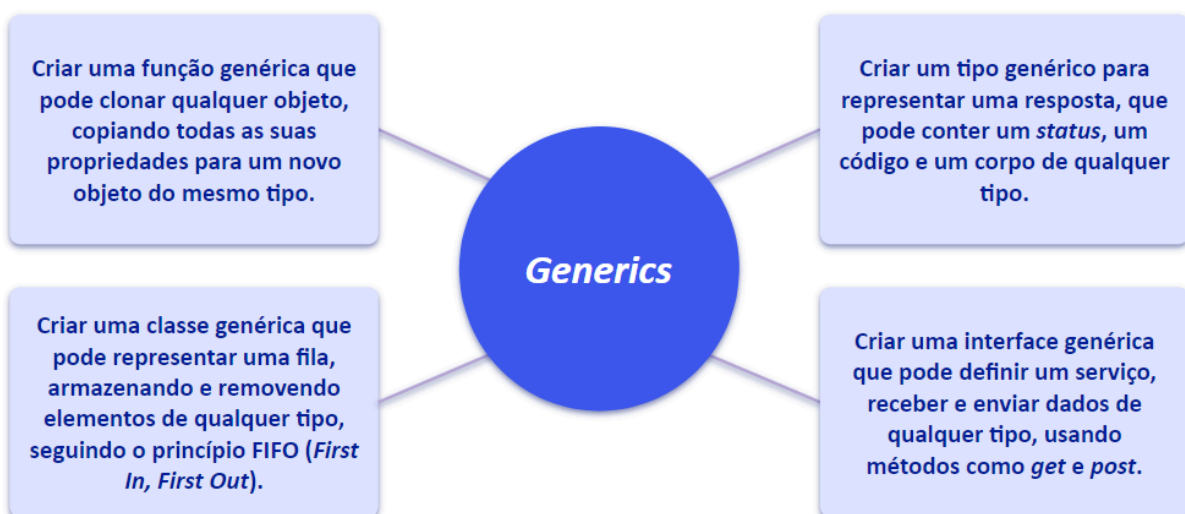
console.log(stack.pop()); // Desempilha e imprime a string "c"

console.log(stack.peek()); // Imprime a string "b", sem desempilhar

console.log(stack.length()); // Imprime o número 2, que é o tamanho da pilha
```

Mas também pode criar uma pilha de outros tipos, como objetos, classes ou interfaces usando o tipo genérico correspondente.

## Aplicações práticas do *generics* em TypeScript



## 10.3 Restrições de tipos com *extends*

Restrições de tipos são uma forma de limitar os tipos aceitáveis por um *generic*. Em TypeScript, você pode usar a cláusula ***extends*** para **definir restrições de tipos, especificando um tipo base que o *generic* deve ser compatível.**

A cláusula *extends* permite que você use as propriedades e métodos do tipo base dentro do *generic*, garantindo a segurança e a consistência do tipo. Por exemplo, para **definir uma função genérica chamada *printName* que recebe um parâmetro do tipo *T*,**

que deve ser um objeto que tenha uma **propriedade *name* do tipo *string***, você pode **usar a cláusula *extends* para restringir o tipo T a ser compatível com o tipo `{ name: string }`**, da seguinte forma:

```
// Define uma função genérica chamada printName que recebe um parâmetro do
// tipo T

function printName<T>(obj: T) {

 // Usa a cláusula extends para restringir o tipo T a ser compatível com o
 // tipo {name: string }

 // Isso garante que o parâmetro obj tenha uma propriedade name do tipo
 // string

 obj extends { name: string };

 // Imprime o valor da propriedade name do parâmetro obj

 console.log(obj.name);

}
```

Com essa definição, você pode **chamar a função `printName` com qualquer objeto que tenha uma propriedade *name* do tipo *string***, como por exemplo:

```
// Cria um objeto chamado user que tem uma propriedade name do tipo string

let user = { name: "Alice", age: 25 };

// Chama a função printName com o objeto user

// O tipo T é inferido como { name: string, age: number }

printName(user); // Alice
```

No entanto, se você tentar chamar a função `printName` com um objeto que não tenha uma propriedade *name* do tipo *string* ou uma variável com um tipo diferente, você

vai receber um erro de compilação. Por exemplo, se você tentar fazer isso:

```
// Cria um objeto chamado product que não tem uma propriedade name do tipo
string

let product = { id: 1, price: 100 };

// Chama a função printName com o objeto product

// O tipo T é inferido como { id: number, price: number }

printName(product); // Erro: O tipo '{ id: number; price: number; }' não é
atribuível ao tipo '{ name: string; }'.

ou

// Cria uma variável chamada num do tipo number

let num = 42;

// Chama a função printName com a variável num

// O tipo T é inferido como number

printName(num); // Erro: O tipo 'number' não é atribuível ao tipo '{ name:
string; }'.
```

**Você vai ver mensagens de erro como estas:**

```
Type '{ id: number; price: number; }' is not assignable to type '{ name:
string; }'.

 Property 'name' is missing in type '{ id: number; price: number; }' but
required in type '{ name: string; }'.

Type 'number' is not assignable to type '{ name: string; }'.
```

Essas mensagens indicam que **os tipos passados como argumentos para a função `printName` não são compatíveis com o tipo base `{ name: string }`** e, por isso, violam a restrição de tipo definida pela cláusula *extends*.

Para fixar o conteúdo, pratique desenvolvendo esses desafios práticos.

## **DESAFIO PRÁTICO**

### Generics in TypeScript.

---

#### **Descrição**

Na atual fase de desenvolvimento, surge a necessidade de implementar uma função genérica chamada **min** que será amplamente utilizada em nosso sistema. Essa função deverá receber dois parâmetros do tipo `T`, os quais devem ser comparáveis, e retornar o menor deles. Para garantir a compatibilidade dos tipos, é importante fazer uso da cláusula *extends*, restringindo o tipo `T` para ser compatível com uma interface que contém um método `compareTo(other: T): number`. Este método `compareTo` deverá retornar um número negativo, zero ou positivo, dependendo se o objeto é menor, igual ou maior que o outro, respectivamente. Essa implementação garantirá a consistência e segurança de tipos em nosso código, contribuindo para a robustez e manutenção do sistema.

---

#### **Objetivos**

- Usar a cláusula *extends* para definir restrições de tipos em TypeScript;
  - Usar as propriedades e métodos do tipo base dentro do *generic*;
  - Criar uma função genérica que pode receber e retornar valores de qualquer tipo que seja comparável.
-

## Orientações

- Crie um arquivo chamado `min.ts` e defina uma função genérica que recebe dois parâmetros do tipo `T`;
- Use a cláusula `extends` para restringir o tipo `T` e dentro do corpo da função, compare os dois parâmetros usando o método `compareTo` e retorne o menor deles;
- Salve o arquivo e teste a sua função com alguns valores de tipos diferentes que sejam comparáveis, como números, *strings*, ou objetos personalizados que implementem o método `compareTo`.



## DESAFIO PRÁTICO

### Generics in TypeScript



#### Descrição

Em nossa equipe de desenvolvimento, surge a necessidade de criar uma classe genérica chamada `ArrayWrapper` para encapsular e manipular *arrays* de elementos de forma mais eficiente e segura. Esta classe receberá um tipo `T` como parâmetro e terá uma propriedade privada denominada *array*, a qual armazenará um *array* de elementos do tipo `T`. Além disso, definiremos um construtor que receberá um *array* de elementos do tipo `T` e o atribuirá à propriedade *array*. Por fim, implementaremos um método público chamado `getArray`, que será responsável por retornar o *array* armazenado na propriedade *array*. Essa classe facilitará a manipulação de *arrays* em nosso código, promovendo a reutilização e a manutenção das implementações.



#### Objetivos

- Utilizar *generics* em TypeScript para criar classes que podem ter propriedades e métodos de qualquer tipo;

- Usar o tipo T como parâmetro para definir o tipo dos elementos do *array*;
- Definir um construtor que recebe um *array* de elementos do tipo T e atribui ele à propriedade *array*;
- Definir um método público que retorna a propriedade *array*.

---

## Orientações

- Crie um arquivo chamado `array-wrapper.ts` e defina uma classe genérica chamada `ArrayWrapper` que recebe um tipo T como parâmetro;
- Defina uma propriedade privada chamada `array` que armazena um array de elementos do tipo T e também um construtor que recebe um array de elementos do tipo T como parâmetro;
- Atribua o parâmetro recebido à propriedade `array` e defina um método público que retorna a propriedade `array`;
- Salve o arquivo e teste a sua classe com alguns arrays de tipos diferentes, como números, strings ou objetos personalizados, criando instâncias da classe.



*Generics* são uma forma de criar elementos que podem trabalhar com diferentes tipos de dados, sem perder a segurança e a consistência do tipo. Em TypeScript, você pode usar *generics* para criar funções, classes, interfaces ou tipos que podem receber e retornar valores de qualquer tipo, desde que esse tipo seja compatível com as restrições de tipo definidas pelo *generic*.

Você também pode usar a cláusula *extends* para definir restrições de tipos para os *generics*, especificando um tipo base que o *generic* deve ser compatível. Isso permite que você use as propriedades e métodos do tipo base dentro do *generic*, garantindo a segurança e a consistência do tipo.



## ATIVIDADE DE FIXAÇÃO

1. Crie uma função genérica chamada *reverse* que recebe um *array* de elementos do tipo *T* e retorna um novo *array* com os elementos na ordem inversa.
2. Crie uma classe genérica chamada *Queue* que recebe um tipo *T* como parâmetro e tem uma propriedade privada chamada *items* que armazena um *array* de elementos do tipo *T*. Em seguida, defina um construtor que inicializa a propriedade *items* como um *array* vazio. Por fim, defina os seguintes métodos públicos: *enqueue*, que recebe um elemento do tipo *T* e adiciona ele no final da fila; *dequeue*, que remove e retorna o elemento do início da fila; *front*, que retorna o elemento do início da fila sem remover ele; *isEmpty*, que retorna um valor booleano indicando se a fila está vazia ou não; e *size*, que retorna o número de elementos na fila.
3. Crie uma interface genérica chamada **Animal** que recebe um tipo *T* como parâmetro e define as seguintes propriedades: *name*, do tipo *string*; *age*, do tipo *number*; e *sound*, do tipo *T*. Em seguida, crie duas classes que implementam essa interface: **Dog**, que recebe o tipo *string* como parâmetro e define o som como "woof"; e **Cat**, que recebe o tipo *boolean* como parâmetro e define o som como *true*.
4. Crie um tipo genérico chamado *Result* que recebe dois tipos *T* e *U* como parâmetros e representa um objeto que pode ter uma das seguintes formas: { *success*: *true*, *value*: *T* }, se a operação foi bem-sucedida e tem um valor do tipo *T*; ou { *success*: *false*, *error*: *U* }, se a operação falhou e tem um erro do tipo *U*.
5. Crie uma função genérica chamada *divide* que recebe dois números como parâmetros e retorna um valor do tipo *Result<number, string>*. Se o segundo parâmetro for diferente de zero, a função deve retornar um objeto com a propriedade *success* igual a *true* e a propriedade *value* igual ao resultado da divisão dos dois números. Se o segundo parâmetro for igual a zero, a função deve retornar um objeto com a propriedade *success* igual a *false* e a propriedade *error* igual a "Division by zero".



6. Crie uma classe genérica chamada **Matrix** que recebe um tipo **T** como parâmetro e tem uma propriedade privada chamada *data*, que armazena um *array* bidimensional de elementos do tipo **T**. Em seguida, defina um construtor que recebe um número de linhas e um número de colunas como parâmetros e inicializa a propriedade *data* como um *array* vazio com o tamanho especificado. Por fim, defina os seguintes métodos públicos: *get*, que recebe um índice de linha e um índice de coluna e retorna o elemento correspondente na matriz; *set*, que recebe um índice de linha, um índice de coluna e um valor do tipo **T**, atribuindo esse valor ao elemento correspondente na matriz; e *print*, que imprime a matriz no console.
7. Crie uma interface genérica chamada **Callback** que recebe dois tipos **T** e **U** como parâmetros e representa uma função que recebe um valor do tipo **T** e retorna um valor do tipo **U**. Em seguida, crie uma função genérica chamada *map* que recebe um *array* de elementos do tipo **T** e uma função do tipo **Callback<T, U>** como parâmetros e retorna um novo *array* com os elementos transformados pela função.
8. Crie uma classe genérica chamada **Node** que recebe um tipo **T** como parâmetro e tem duas propriedades públicas: *value*, que armazena um valor do tipo **T**; e *next*, que armazena uma referência para outro **Node** do mesmo tipo ou *null*. Em seguida, defina um construtor que recebe um valor do tipo **T** como parâmetro e inicializa as propriedades *value* e *next*.
9. Crie uma interface genérica chamada **Iterable** que recebe um tipo **T** como parâmetro e define um método chamado *iterator*, que retorna um objeto que implementa a interface **Iterator<T>**. Em seguida, crie uma interface genérica chamada **Iterator** que recebe um tipo **T** como parâmetro e define um método chamado *next*, que retorna um objeto que tem duas propriedades: *value*, do tipo **T**; e *done*, do tipo *boolean*.
10. Crie uma classe genérica chamada **LinkedList** que recebe um tipo **T** como parâmetro e implementa a interface **Iterable<T>**. A classe deve ter uma propriedade privada chamada *head*, que armazena uma referência para o primeiro **Node<T>** da lista ou *null*. Em seguida, defina um construtor que inicializa

a propriedade *head* como *null*. Por fim, defina os seguintes métodos públicos: *append*, que recebe um valor do tipo T e adiciona um novo Node<T> no final da lista; *prepend*, que recebe um valor do tipo T e adiciona um novo Node<T> no início da lista; *remove*, que recebe um valor do tipo T e remove o primeiro Node<T> que contém esse valor da lista; *contains*, que recebe um valor do tipo T e retorna um valor booleano indicando se a lista contém um Node<T> com esse valor; e *iterator*, que retorna um objeto que implementa a interface Iterator<T> e permite percorrer os elementos da lista.

## Referências

**FLATSCHART**, Fábio. Html 5: embarque imediato. 1. ed. Rio de Janeiro: Brasport, 2011. E-book. Disponível em: <https://plataforma.bvirtual.com.br> Acesso em: 16 nov. 2023.

**GOLDBERG**, Josh. Aprendendo TypeScript. [Recurso eletrônico]. Editora Novatec, 2022.

**PAZ**, Mônica. Webdesign. 1. ed. Curitiba: Intersaberes, 2021. E-book. Disponível em: <https://plataforma.bvirtual.com.br> Acesso em: 16 nov. 2023.

**CASTRO**, Bruno; **VIEIRA**, Rui. Python: Introdução à Resolução de Problemas com Algoritmos. Rio de Janeiro: Editora Ciência Moderna, 2012.

**FORBELLONE**, André Luiz Villar; **EBERSPÄCHER**, Henri Frederico. Lógica de Programação: A Construção de Algoritmos e Estruturas de Dados. 4. ed. São Paulo: Pearson Prentice Hall, 2019.

**GAMA**, Eric. Python Fluente: Programação Clara, Concisa e Eficaz. São Paulo: Novatec, 2016.

**GOMES**, Cláudio; **FERNANDES**, Antônio. Lógica de Programação e Estrutura de Dados: Algoritmos em Java. Rio de Janeiro: LTC, 2015.

**LEMONS**, Sandro; **WIEDEMANN**, Nelson. Python: Escreva seus primeiros programas. São Paulo: Digerati Books, 2017.

**LUTZ**, Mark. Aprendendo Python: Do Iniciante ao Profissional. São Paulo: Novatec, 2014.

**MANZANO**, José Augusto N. G.; **OLIVEIRA**, Jayr Figueiredo de. Algoritmos: Lógica para Desenvolvimento de Programação de Computadores. 28. ed. São Paulo: Érica, 2018.

**PRATA**, Bruno. Lógica de Programação: A Construção de Algoritmos e Estruturas de Dados. 1. ed. São Paulo: Casa do Código, 2017.

**VIEIRA**, Sérgio. Python para Desenvolvedores. 2ª edição. São Paulo: Novatec, 2010.

**ZIVIANI**, Nivio. Projeto de Algoritmos com Implementações em Pascal e C. 4. ed. São Paulo: Cengage Learning, 2015.